# 线性表

## Logic

在数据元素的非空有限集中

1. 存在唯一的被称为”第一个“的元素
2. 存在唯一的被称为”第二个“的元素
3. 除第一个元素外，每个元素都只有一个前驱
4. 除最后一个外，每个元素都只有一个后继

## Physic

### 顺序表

```
#define  MAXNUM  100      /* 线性表空间大小 */
#define  FALSE   0
#define  TRUE    1
typedef  int  DataType;                  /*  定义元素类型为整型
                                          ，也可定义为其他类型 */
#define  SPECIAL  2147483647
/* 与DataType 类型有关，32位机上的最大整数为2^31-1 */
struct  SeqList
{
        DataType  element[MAXNUM];
        int  length;
};
typedef  struct SeqList   SeqList, *PSeqList;
```

可变长的顺序表

```
typedef struct SeqList
{
        DataType  *elem;  /* 存储元素连续空间的地址*/
        int  size;          /* 表的大小 */
        int  length;        /* 表中元素的个数*/
}SeqList, *PSeqList;
```

链表

```
typedef int DataType;        /* 定义元素类型为整型，也可定义为其他类型 */
struct  Node                 /* 单链表结点结构 */
{
      DataType  info;
      struct Node  *next;
};
typedef struct Node  Node, *PNode;          /* 结点指针类型 */

struct LinkList                          /* 单链表类型定义 */
{
      PNode  head;                       /* 指向单链表中的第一个结点 */
};
typedef  struct LinkList  LinkList,*PLinkList;    /* 单链表类型的指针类型 */

PLinkList  pllist; /* pllist是指向单链表的一个指针变量 */
```

双向链表

```
typedef struct DoubleNode        /* Node definition */
{
        DataType   info;
        struct DoubleNode  *llink, *rlink;
}DoubleNode, *PDoubleNode;

typedef struct DoubleList        /* 双向链表类型 */
{
        PDoubleNode  head;       /* 指向第一个结点 */
        PDoubleNode  tail;       /* 指向最后一个结点 */
};
PDoubleList  pdlist;             /* pdlist是指向双链表类型的
                                    指针变量 */
```




# Algorithm

## 顺序表

查找：成功平均n/2次 不成功n次

删除：平均移动（n-1）/2次

可变长顺序表创建：

```
/* Create a new blank list */
void InitList( PSeqList pList )
{
    pList->elem = (ElemType *) malloc(InitSize*sizeof(ElemType));
    /* Test whether allocation is successful */
    assert(pList->elem != NULL);

    pList->size  = InitSize;
    pList->length= 0;
}       /* End of InitList() */
```

click

一元多项式

```
typedef  struct  ElemType      /* 二元组*/
{
      float  coef;             /* 多项式系数 */
      int  expn;               /* 多项式指数 */
}ElemType;
/* 结点属性 */

typedef  struct  Polyn
{
      ElemType  data;          /* 数据域 */
      struct Polyn  *next;     /* 指针域 */
}Node, *Link;
/* Node type & Node Pointer type 结点类型及结点指针类型 */
```

# 栈

## Logic

操作受限的线性表

栈顶用于数据的插入和删除，还有一个栈底

## Physic

### 顺序表

```
typedef  int  ElemType;          /* 定义栈元素的数据类型，
                                    这里定义为整型 */
#define  MAXNUM  100             /* 栈中能达到的最大容量，
                                    这里设为100 */
typedef  struct  SeqStack        /* 顺序栈类型定义 */
{
      ElemType  s[MAXNUM];
      int  top; /* 栈顶指针 */
}SeqStack, *PSeqStack;
```

### 链表

```
typedef  int  DataType ;        /* 定义栈中元素类型为整型，
                                   也可定义为其他类型 */
struct  Node                    /* 单链表结点结构 */
{
        DataType info;
        struct Node  *link;
};
struct LinkStack                /* 链接栈类型定义 */
{
        struct Node  *top;      /* 指向栈顶结点 */
}LinkStack, *PLinkStack;
```

# Algorithm

## 顺序表

初始化，注意栈顶指针为-1，这也是判空条件

## 链表

初始化栈顶指针为空，这也是判空条件

## 括号匹配

凡出现左括号，则进栈；

凡出现右括号，则首先看栈空不空，若空则出问题，不空则和栈顶比较，匹配到左括号，则栈顶元素出栈，否则不匹配，出问题

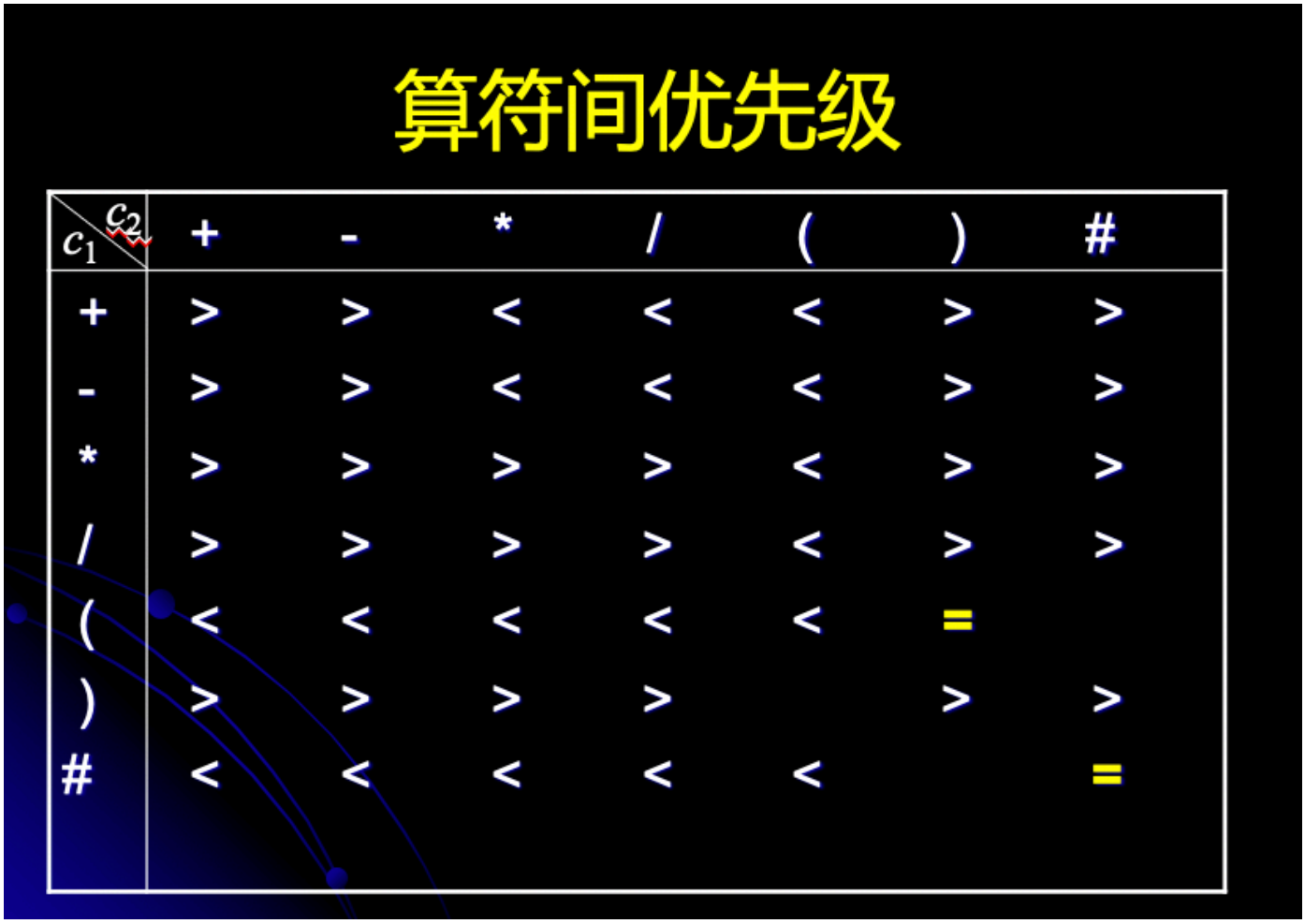
顺利检查完整个表达式后，栈是空的，就对了，不空，还是出问题

## 中缀表达式求值

首先定义算符的优先级：

括号优先级最高，对括号内的运算，乘除后加减，同级从左至右

每一个运算步，比较相邻的算符c1，c2。有这个表




接着是具体做法：

一个OPTR放算符，一个OPND放数据

首先把栈顶放一个#，表达式最后放一个#

自左向右扫描表达式，操作数进OPND，操作码c2则和OPTR的顶元素c1比较，三种情况

1. c1>c2 说明栈里那个元素要先算，把它出栈，从OPND取顶元素运算
2. c1=c2说明右括号遇到了左括号，把c1出栈
3. c1<c2说明栈里的元素比现在的元素后算，继续读表达式

重复至表达式求完

## 后缀表达式求值

一个数据栈，读到操作码就把头两个出栈算出结果入栈

# 队列

## Logic

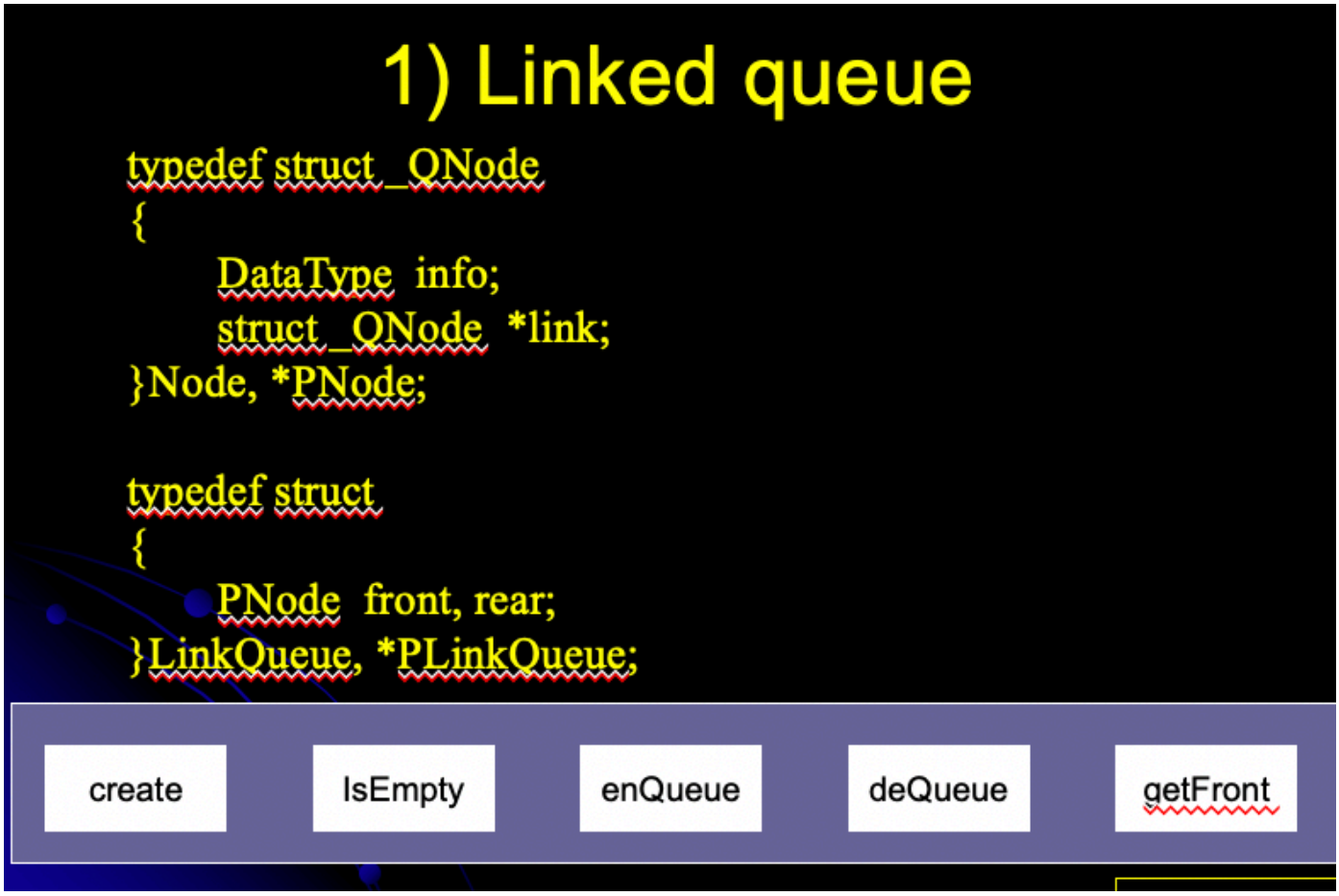
特殊的线性表，插入在一端，删除在另一端，删除是头，插入是尾

## Physic

### 顺序队列

```
#define MAXNUM  100          /* 队列中能达到的最大容量，这
                             里设为100 */
struct  SeqQueue             /* 顺序队列类型定义 */
{
        DataType q[MAXNUM];
        int front, rear;
};
/* 顺序队列类型的指针类型 */
typedef  struct SeqQueue  *PSeqQueue;
```

### 链队列




结点指针域只有一个link

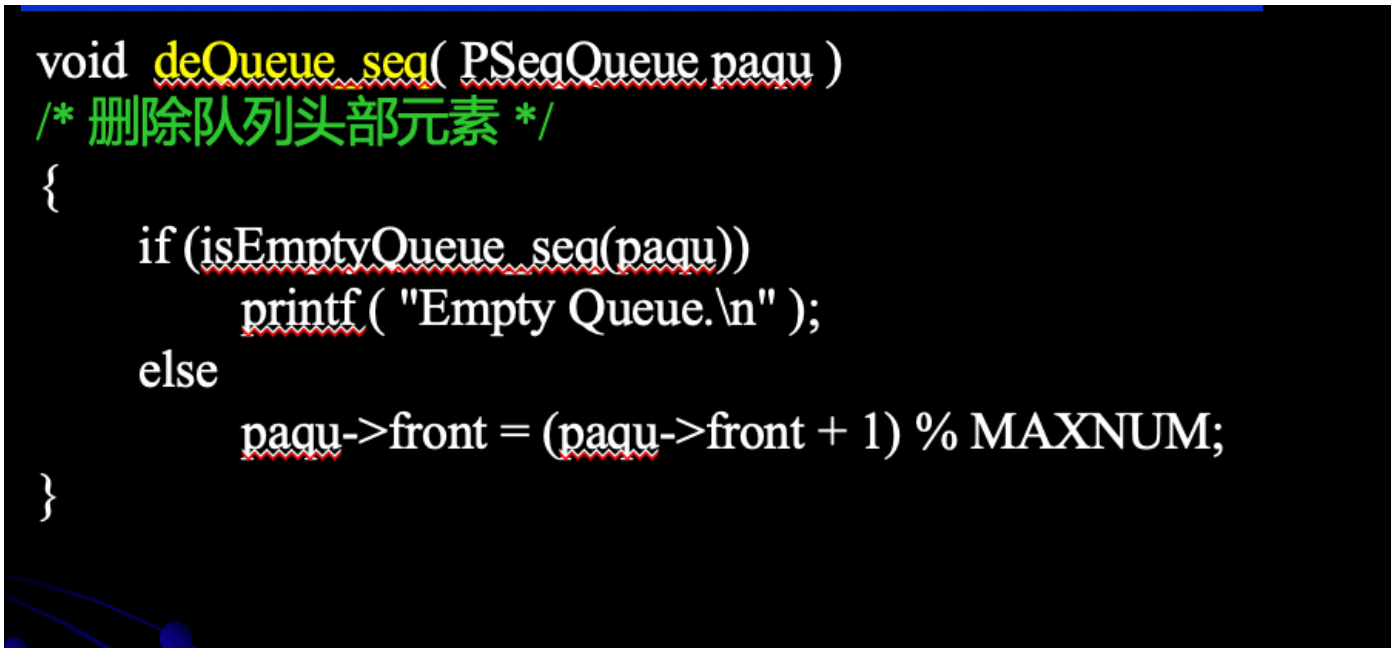
# Algorithm

## 顺序队列

插入，注意mode最大容量

```
void enQueue_seq( PSeqQueue paqu, DataType x )
/* 在队列尾部插入一元素x */
{
    if ( (paqu->rear + 1) % MAXNUM == paqu->front )
        printf ( "Full queue.\n" );
    else {
        paqu->q[paqu->rear] = x;
        paqu->rear = (paqu->rear + 1) % MAXNUM;
    }
}
```

注意，删除也是头加一，因为头在左边，尾在右边



```
void deQueue_seq( PSeqQueue paqu )
/* 删除队列头部元素 */
{
    if (isEmptyQueue_seq(paqu))
        printf( "Empty Queue.\n" );
    else
        paqu->front = (paqu->front + 1) % MAXNUM;
}
```

## 链队列

初始化全空，插入第一个值的时候头尾都是它，之后在尾巴上插，这样看，每个元素只看到自己后面的

## 斐波那契数列

每次求和入队，用循环队列

## 杨辉三角

类似

# 字符串

## Logic

零或多个**字符**组成的有限序列

子串在主串中的位置以子串第一个字符的位置表示

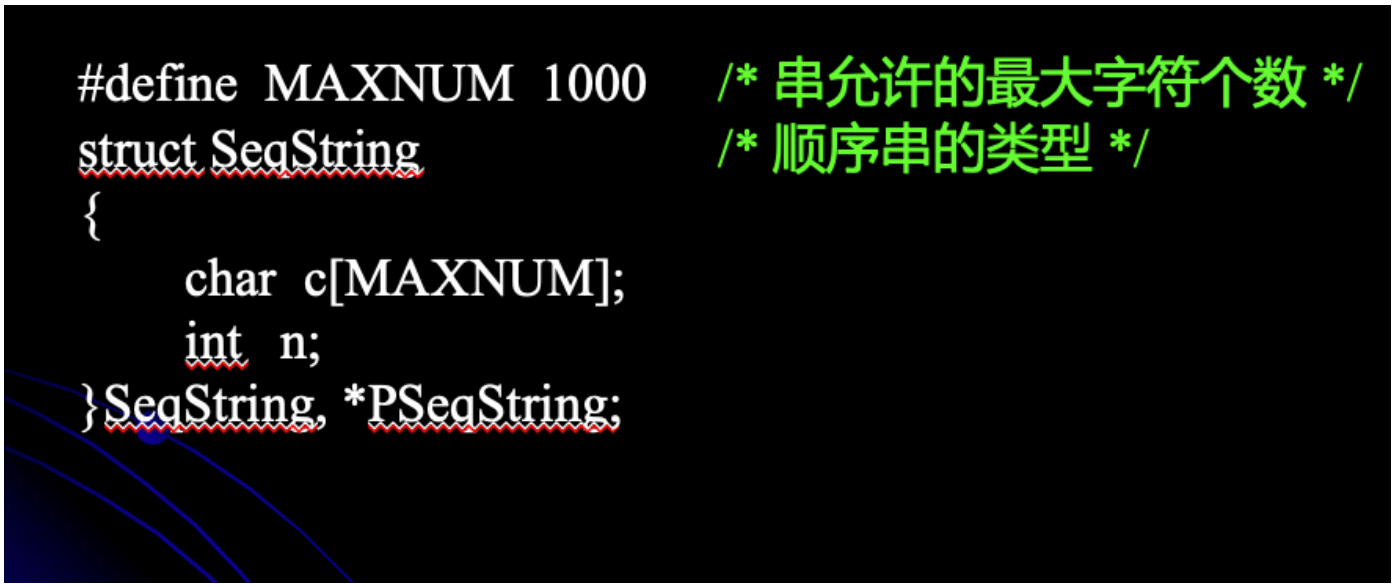
子串是连续的

串的操作以“串的整体”作为操作对象

## Physic

### 顺序

定长顺序表示




变长顺序表示

```
typedef struct
{
        char    *ch;
        int     length;
}HString;
```

**链式**

链表略

块链

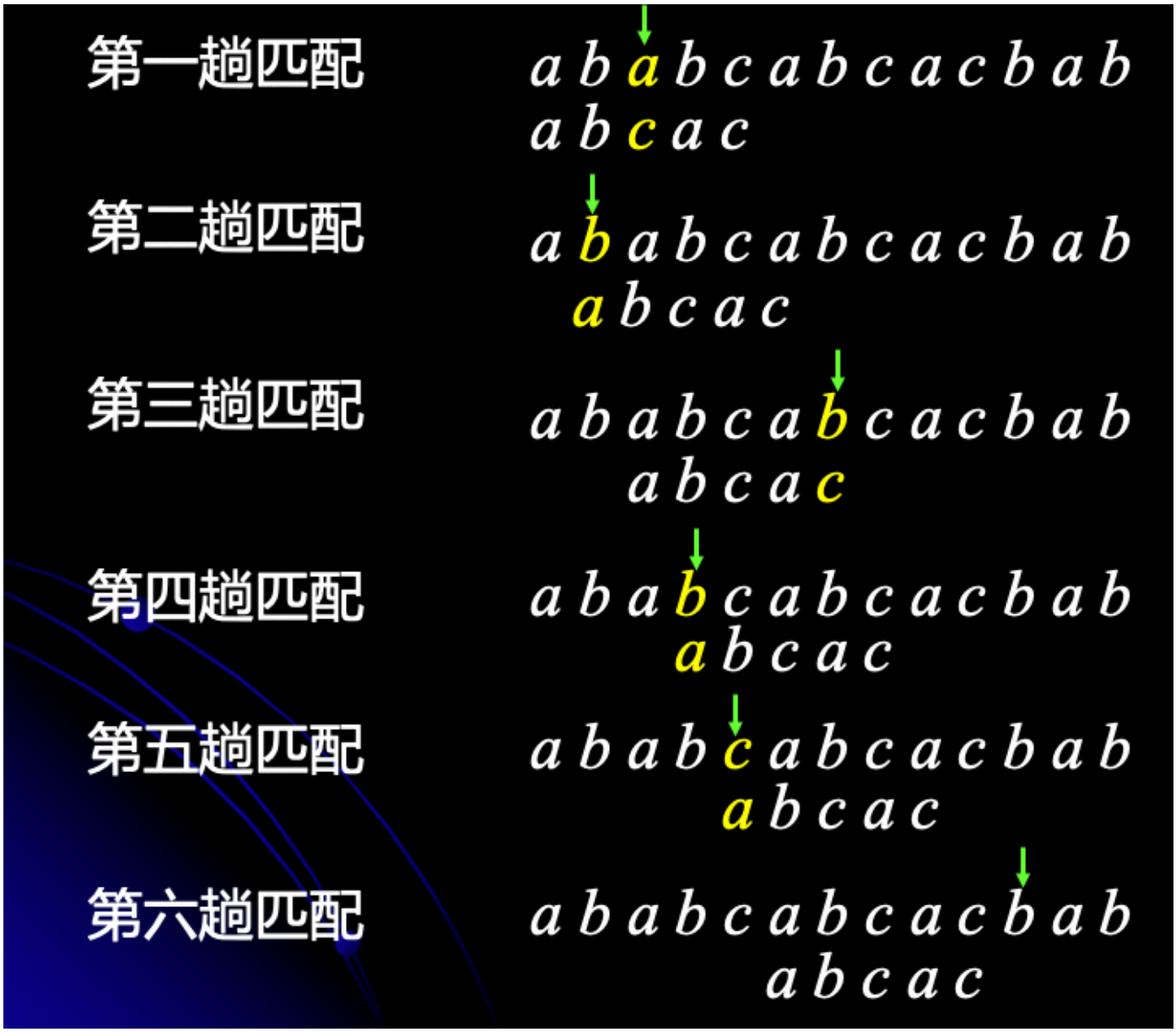
```
#define ChunkSize 80
typedef struct _StrNode
{
    char  c[ChunkSize];
    struct _StrNode  *link;
}StrNode;

typedef struct
{
    StrNode  *head;
}LinkString, *PLinkString;
```

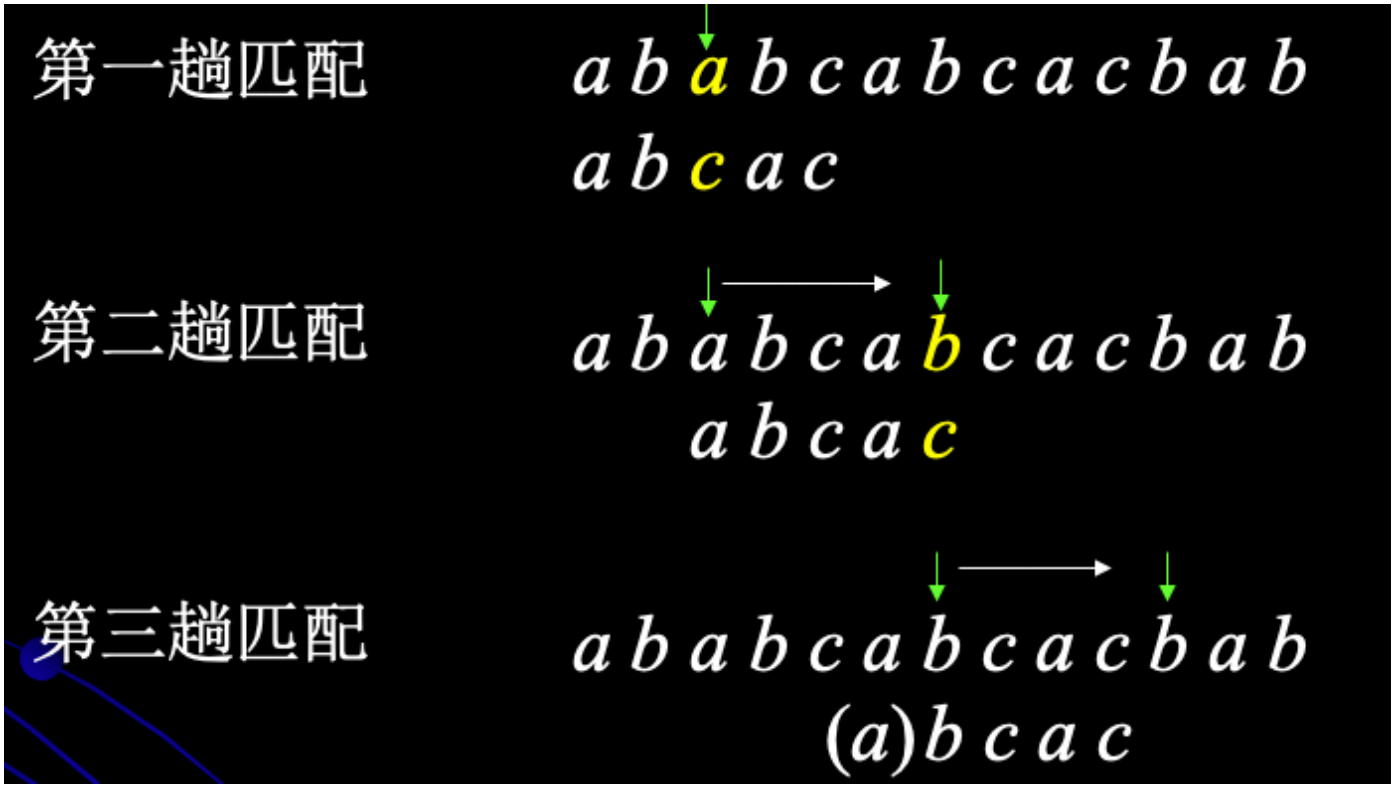
# Algorithm

## 模式匹配

传统算法

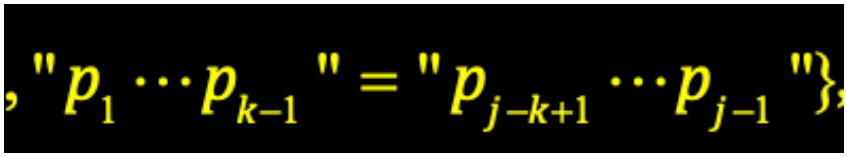



KMP算法




每当不匹配发生，按这种策略去决定模式串右移多少（实际上就是猪串的失配位置下一次和模式串的哪个位置做比较）

再进行一次模式匹配，这次在模式串中找，有没有



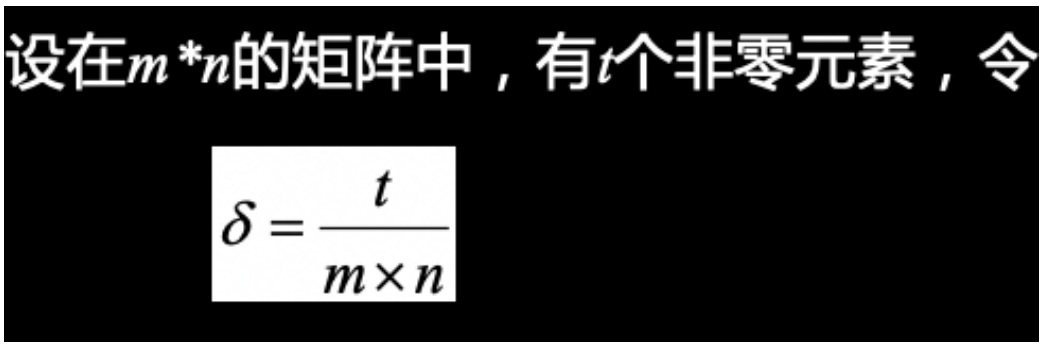

的情况

如果找到了k，就右移k位，不然就乖乖右移一位。

# 稀疏数组

## Logic

稀疏因子

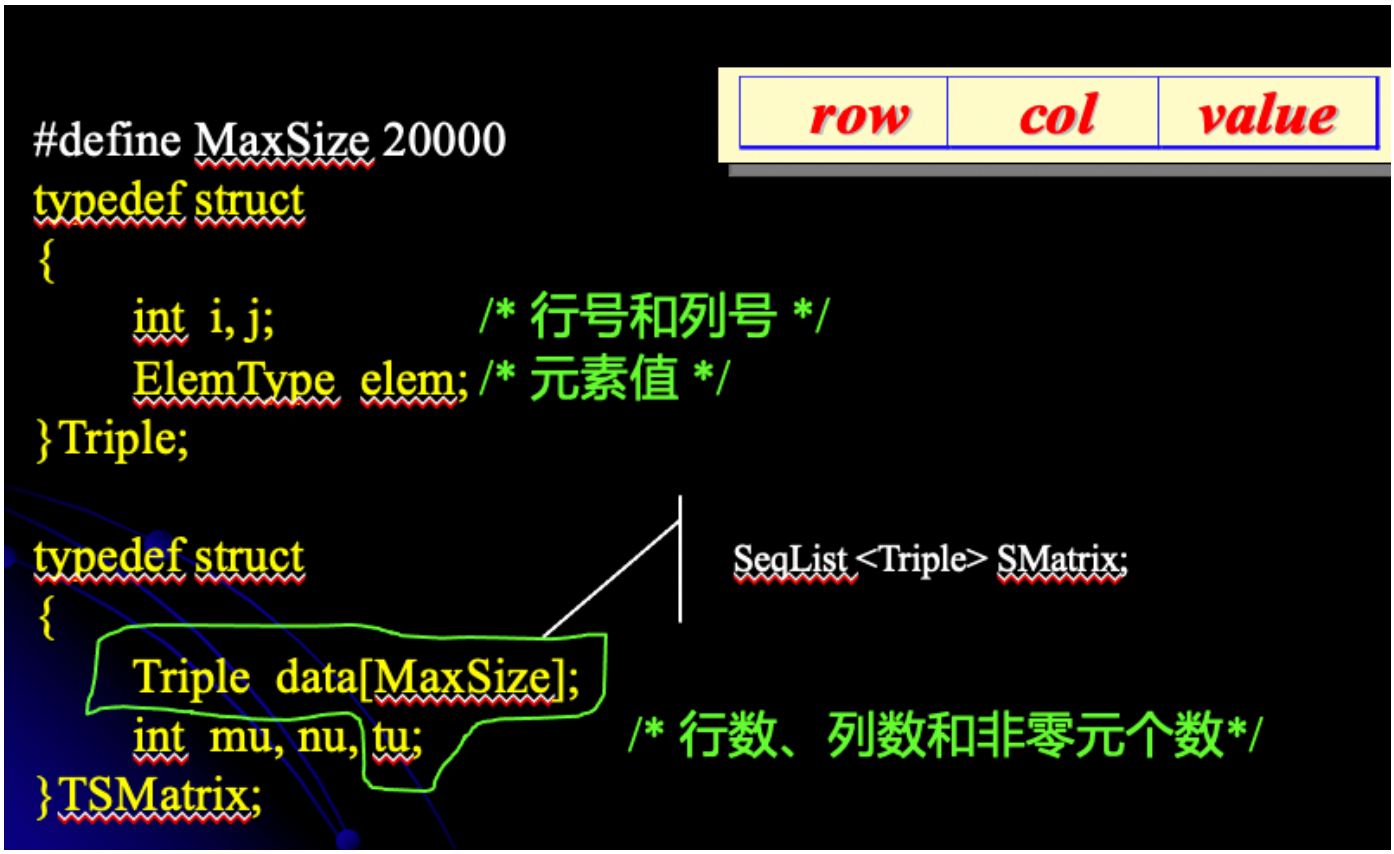



比0.05小的时候，认为是稀疏矩阵

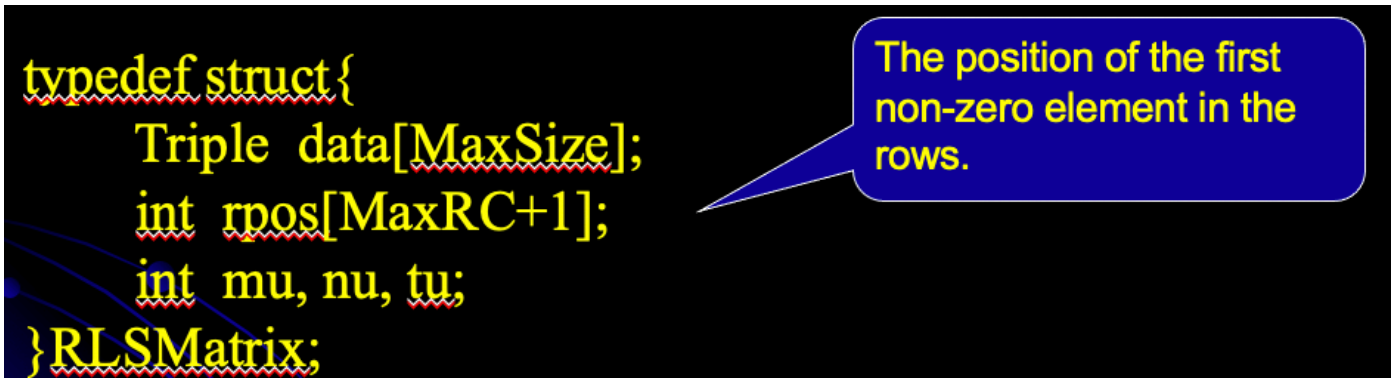
可以用三元组表示

## Physic

顺序三元组表，一个triple一个triple的数组，注意记录行列和非零元个数

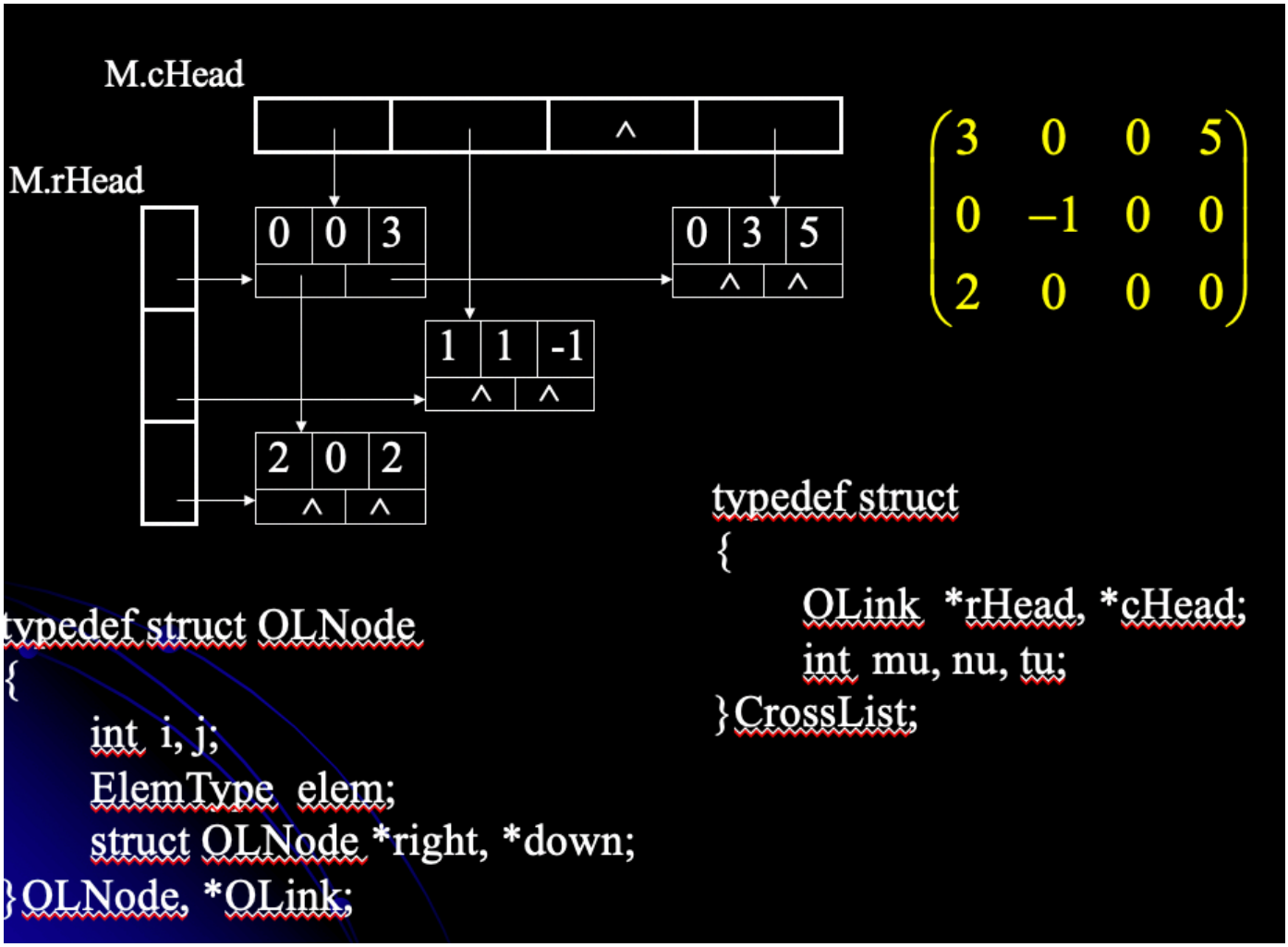


行逻辑链接的顺序表，额外记录每一行第一个非零元在这个三元组表中的索引




十字链表

每一个元素有数据域（三元组）和指针域（右、下），每一个矩阵有数据域（行列数、非零元个数）和指针域（每一行的头指针数组，每一列的头指针数组）

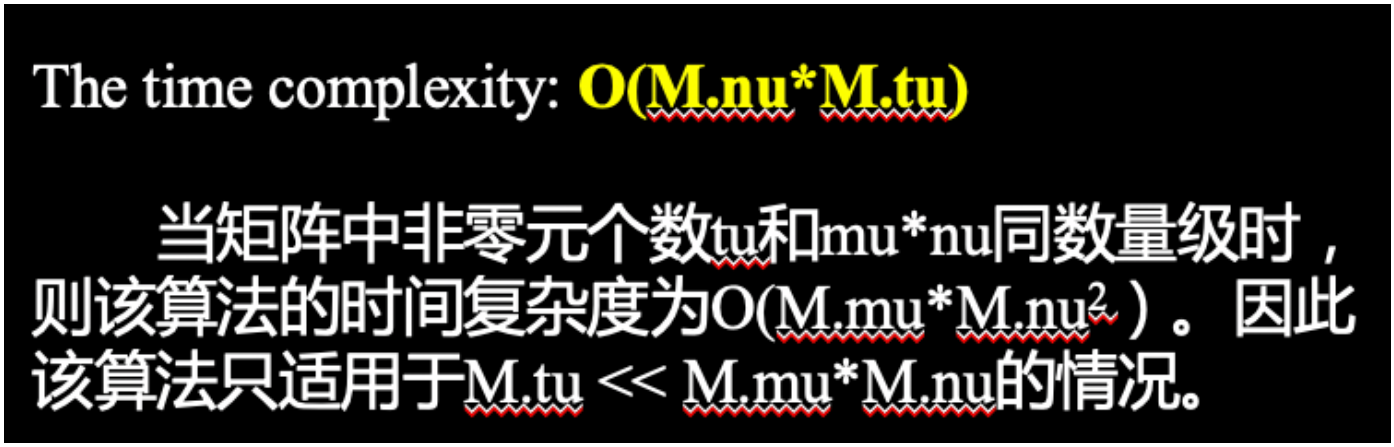


# Algorithm

## 矩阵转置

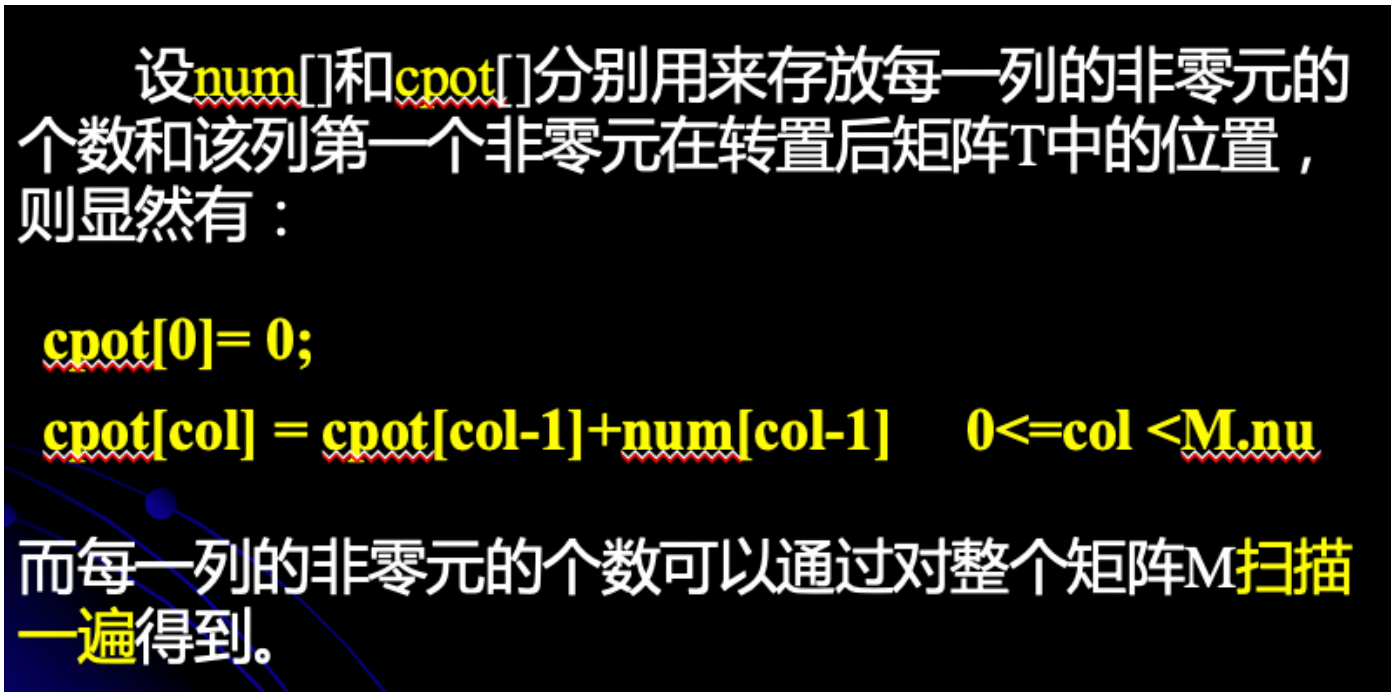
简单算法：

按照列序，从0到最大列依次查找，对于每个列，遍历所有元素，把列号相等的元素填入新的矩阵。




快速算法：

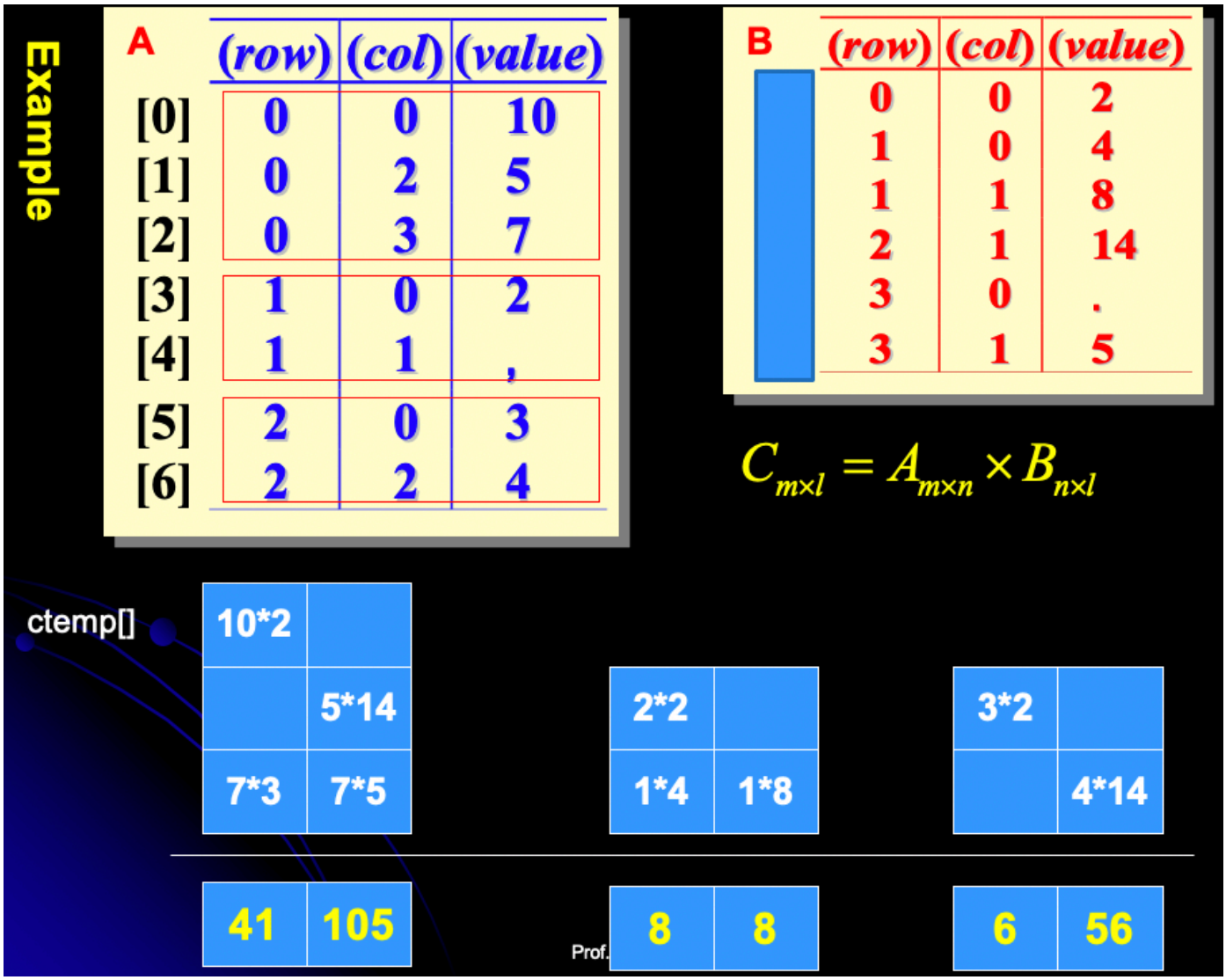
需要两个额外数组，用来预先确定源矩阵中每一列的第一个非零元（第一个意味着行号在这一列里最小，且从它往下行号递增）在**转置后矩阵**中的位置，这样在转置时，我们只要把这一列的元素从记录好的位置依此往下存就行了。



## 矩阵乘法

采用“行逻辑链接的顺序表”结构

对于左边的矩阵，一次取遍同一行的非零元素（因为数据结构的支持，这是很方便的），对于每一个这种元素，找到右边矩阵中行号和它的列号相等的非零元素（这也是很方便的），把乘积累加到对应行列中，遍历完一行，得到结果矩阵的这一行

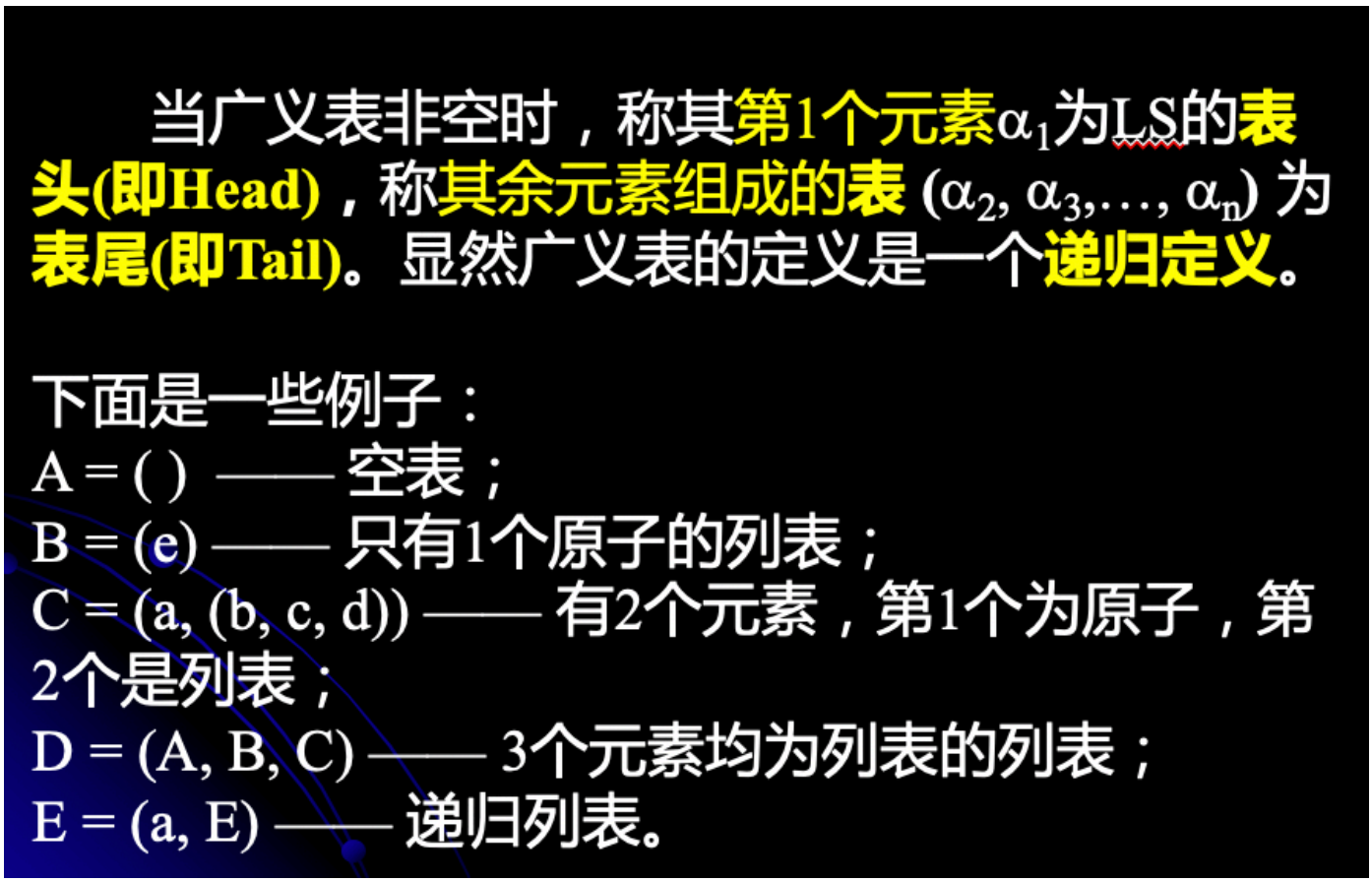



图不是很好看，看字。

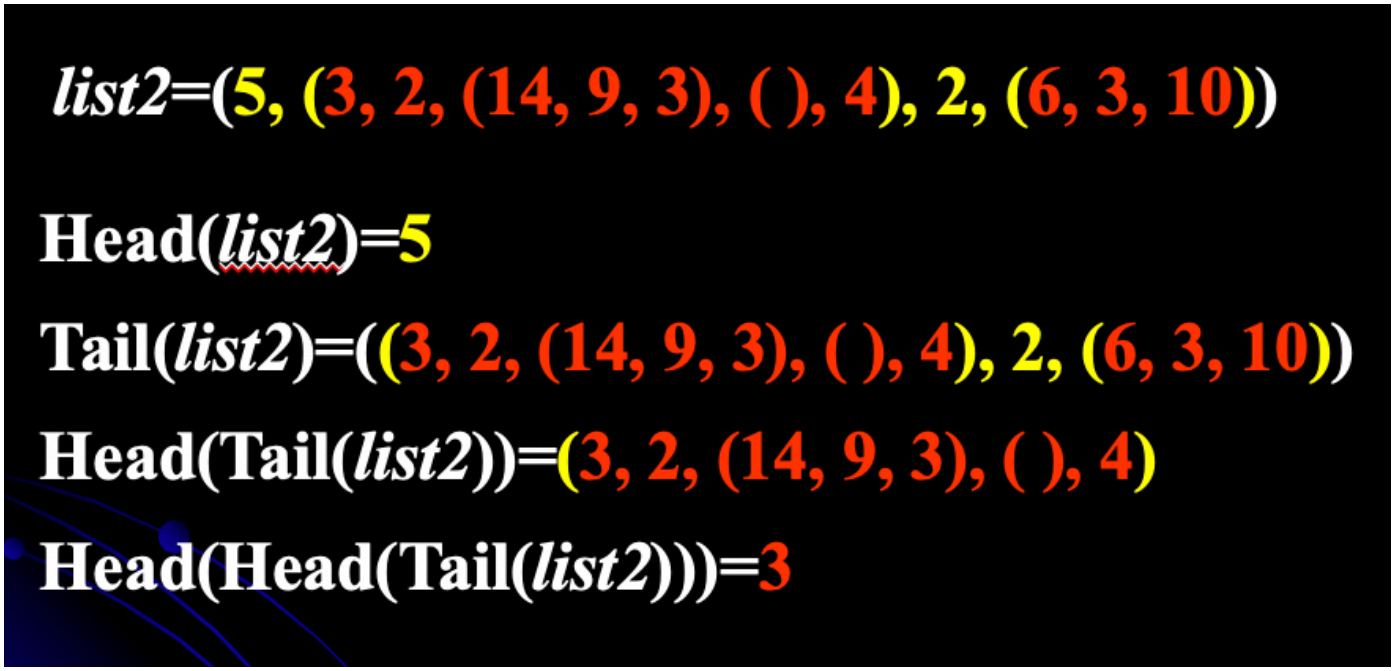
# 广义表

## Logic

广义表是线性表的推广，特点在于它的元素可以是原子，也可以是子表




注意表头表尾的定义



- 广义表可以是一个递归表
- 表头可以是原子，也可以是子表，表尾一定是子表（因为表尾是把表头去掉后的表）

# Physic

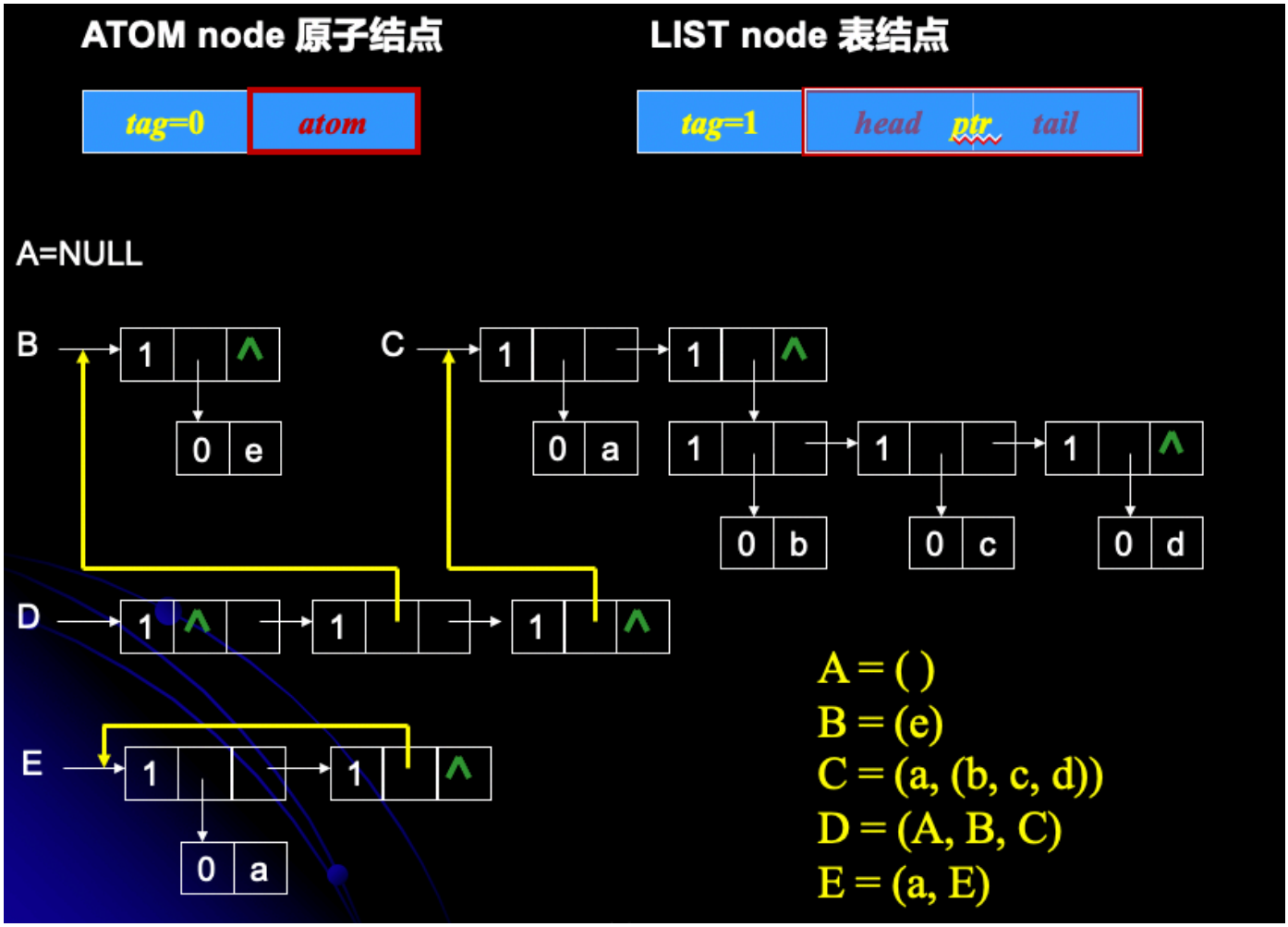
- 表头 表尾法

链式存储，用一个标志域来标记是原子还是子表，这样，表结点需要三个域（标志，表头指针，表尾指针），而原子结点只要两个域（标志，值）。

注意表尾记录的是同层次的后继




标志域是原子和表的共有特性，用一个公用体区分原子和表



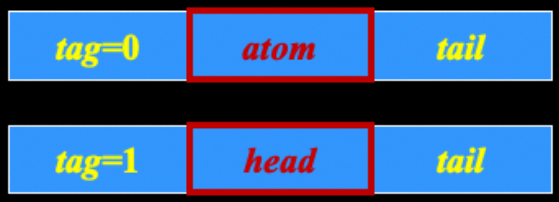

- 头指针、下结点法（注意下结点不是表尾）

每一个结点都可以知道自己的下一个元素，一个表的表头结点的tail指针指向同层次的后继元素。

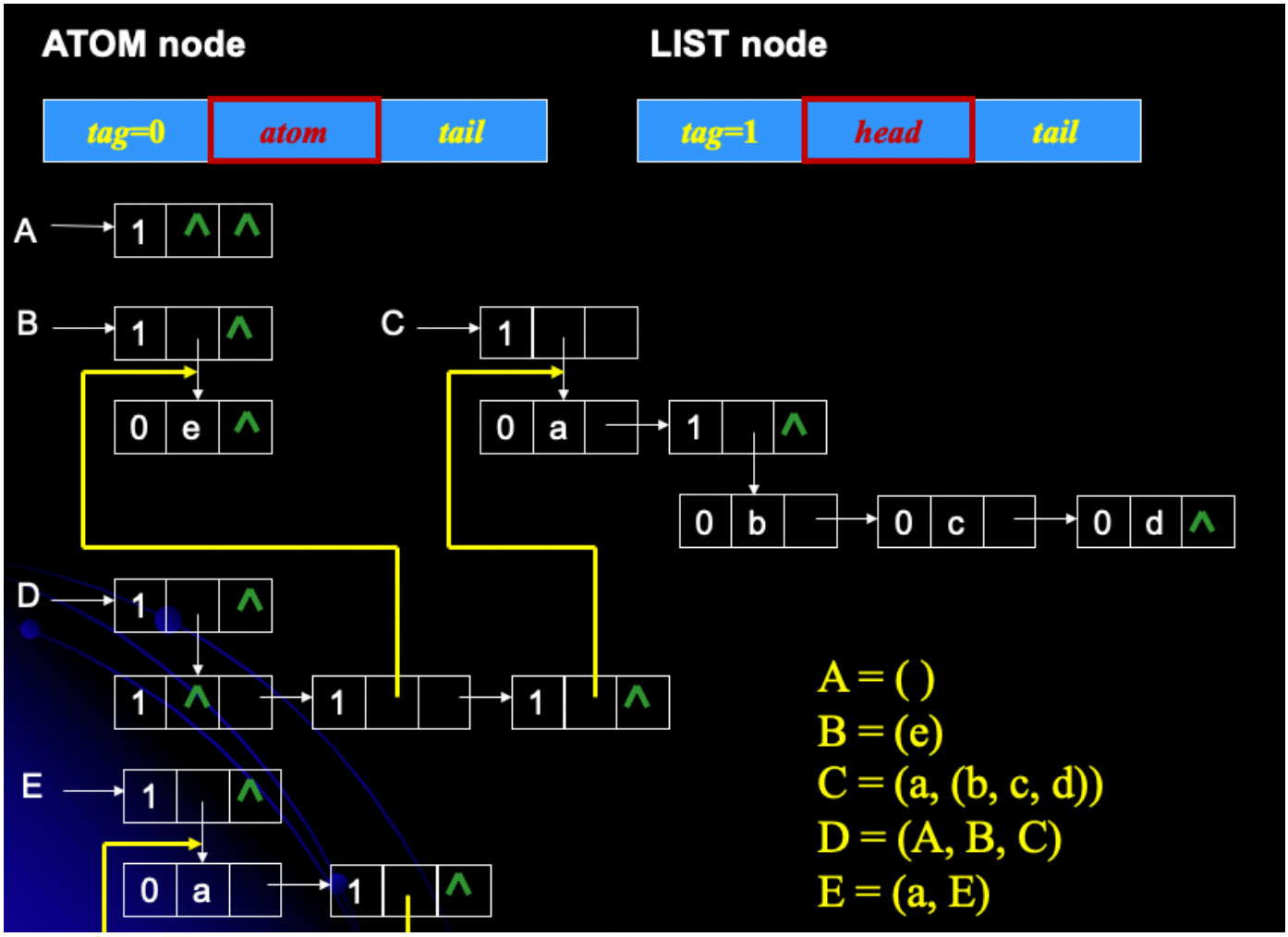
## Alternative representation of General list: 表结点头指针+下一个结点

```
typedef enum {ATOM, LIST} ElemTag;
//ATOM==0 原子; LIST==1子表
typedef int AtomType;

typedef struct _GLNode
{
   ElemTag  tag;
   union
   {
      AtomType  atom;          //原子结点的值域
      struct _GLNode  *head;   //表结点的表头指针
   };
   struct _GLNode  *tail;      //指向下一个元素结点
} *GList;
```




共有一个标志和尾指针，表还知道表头，原子还知道值




# Algorithm

## 广义表的深度

深度的定义：

广义表LS = ($\alpha_1$,$\alpha_2$,…, $\alpha_n$)的深度Depth(LS) 递归定义为：

基本项：Depth(LS) = 1 当LS为空表时

Depth(LS) = 0 当LS为原子时

归纳项：Depth(LS) = 1 + Max{Depth($\alpha_i$)} i=1..n, n>=1

递归计算就行，每次遍历当前表的所有后继，它们表头深度最大值，加一返回。

```
int GListDepth (GList L) {
   if (!L) return 1;  //空表深度为1
   if (L->tag == ATOM) return 0; //原子深度为0
   for ( max=0, pp=L; pp; pp=pp->ptr.tail) {
      dep = GListDepth (pp->ptr.head); //求以为头指针的子表深度
      if (dep > max) max = dep;
   }
   return (max+1);
}
```

**Depth(D) = 1+ Max{ 1, 1, 2} = 3**

**D=(A,B,C)=(() , (e), (a, (b,c,d)))**

Depth(D) = 1+Max{ Depth(A), Depth(B), Depth(C)}

Depth(A) = 1;

Depth(B) = 1+ Max{Depth(e)} = 1+0 = 1;

Depth(C) = 1+ Max{Depth(a), Depth((b,c,d))} = 2

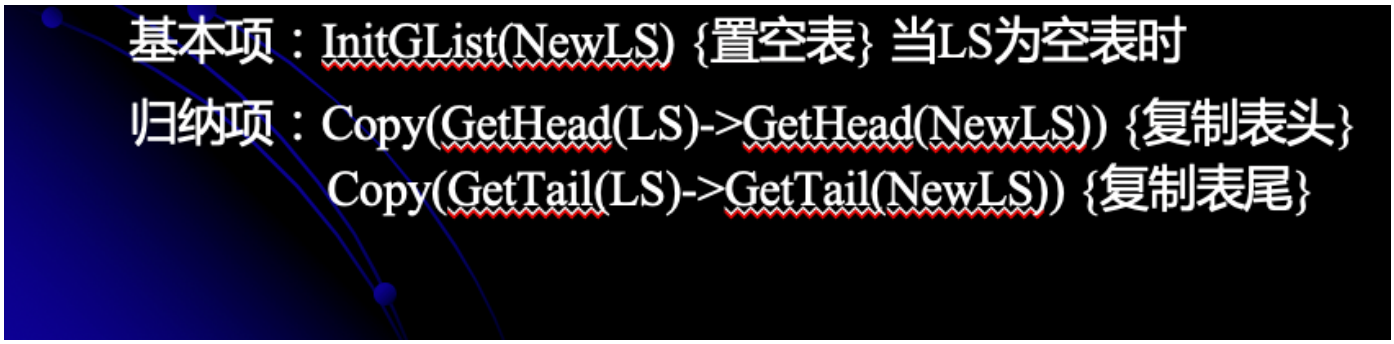
Depth(a) = 0;

Depth ((b,c,d)) = 1+Max{Depth(b), Depth(c), Depth(d)}

= 1+0 = 1;

## 复制广义表

也是递归




```
Status CopyGList (GList T, GList L) {
    if (!L) T = NULL; //复制空表
    else {
        if (!(T = (GList) malloc (sizeof(GLNode)) exit(OVERFLOW);
        T->tag = L->tag;                                    //建表结点
        if (L->tag == ATOM ) //复制单原子
            T->atom = L->atom;
        else {
            CopyGList (T->ptr.head, L->ptr.head);
            // 复制广义表 L->ptr.head 的一个副本 T->ptr.head
            CopyGList (T->ptr.tail, L->ptr.tail);
            // 复制广义表 L->ptr.tail 的一个副本 T->ptr.tail
        }
    }
    return OK;
}
```

# 树

## Logic

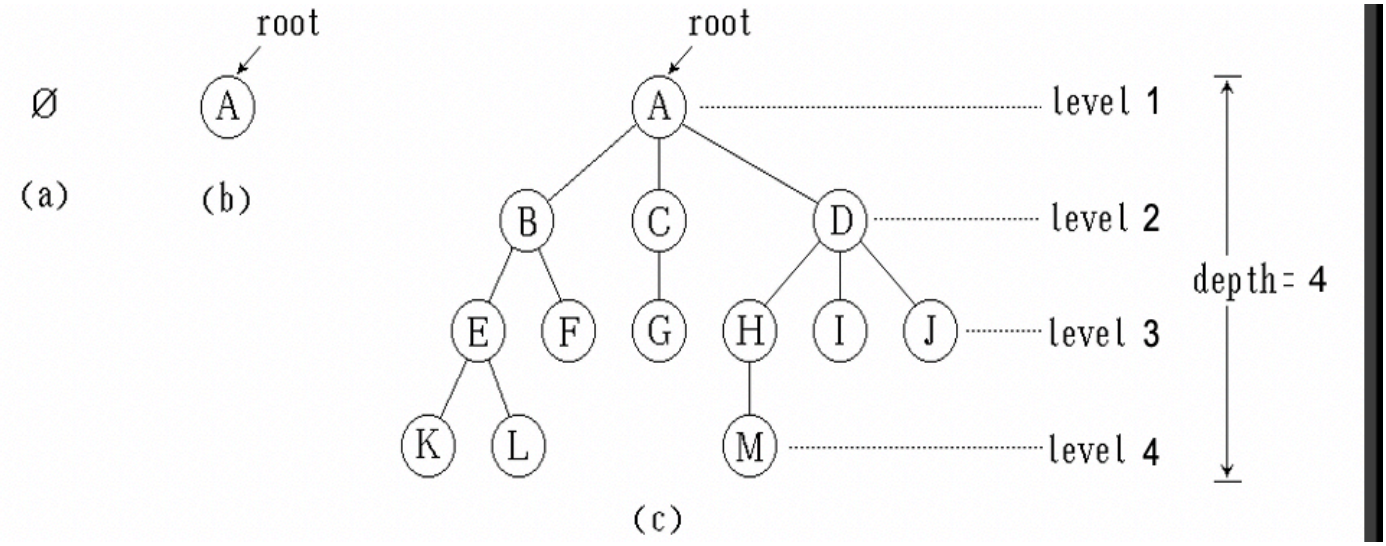
树是n个点的点集，如果n=0，那就是空树，否则，树有一个“根”结点，它只有后继，没有前驱，除了根结点，剩下的部分可以分成m棵树

结点包含一个数据元素和若干指向子树的分支

度是子树个数

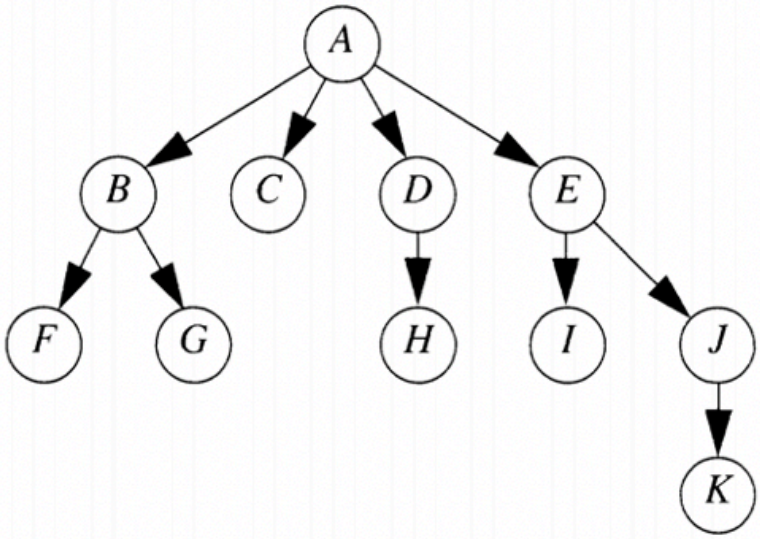
树叶（度为0）、分支结点（度不为0）

层数，根的层数为1




深度高度从0计数

注意：深度和高度从0计数




| Node | Height | Depth |
|---|---|---|
| A | 3 | 0 |
| B | 1 | 1 |
| C | 0 | 1 |
| D | 1 | 1 |
| E | 2 | 1 |
| F | 0 | 2 |
| G | 0 | 2 |
| H | 0 | 2 |
| I | 0 | 2 |
| J | 1 | 2 |
| K | 0 | 3 |

**Figure 18.1** A tree, with height and depth information.

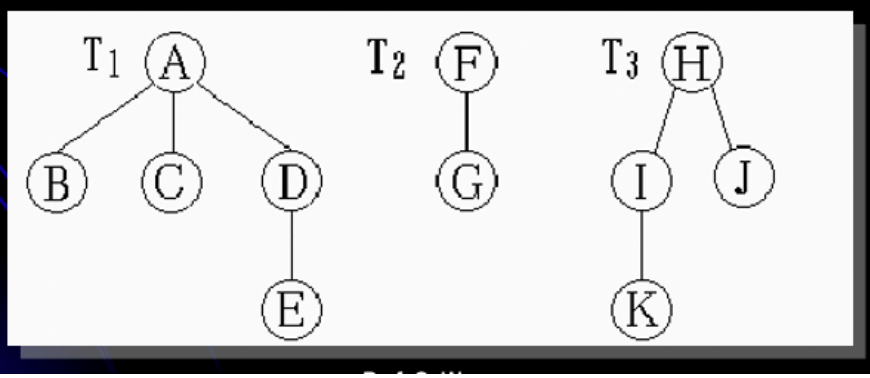
森林

A set of trees. 森林是m (m>=0)棵互不相交的树的集合。对树中每个结点而言，其子树的集合即为森林。

就逻辑结构而言，任何一棵树是一个二元组Tree=(root, F)，其中root称为树的根结点；F是m (m>=0)棵子树构成的森林，$F=(T_1, T_2,\ldots,T_m)$, 其中$T_i=(r_i, F_i)$称作根root的第i棵子树；当m≠0时，在树根和其子树森林之间存在下列关系:

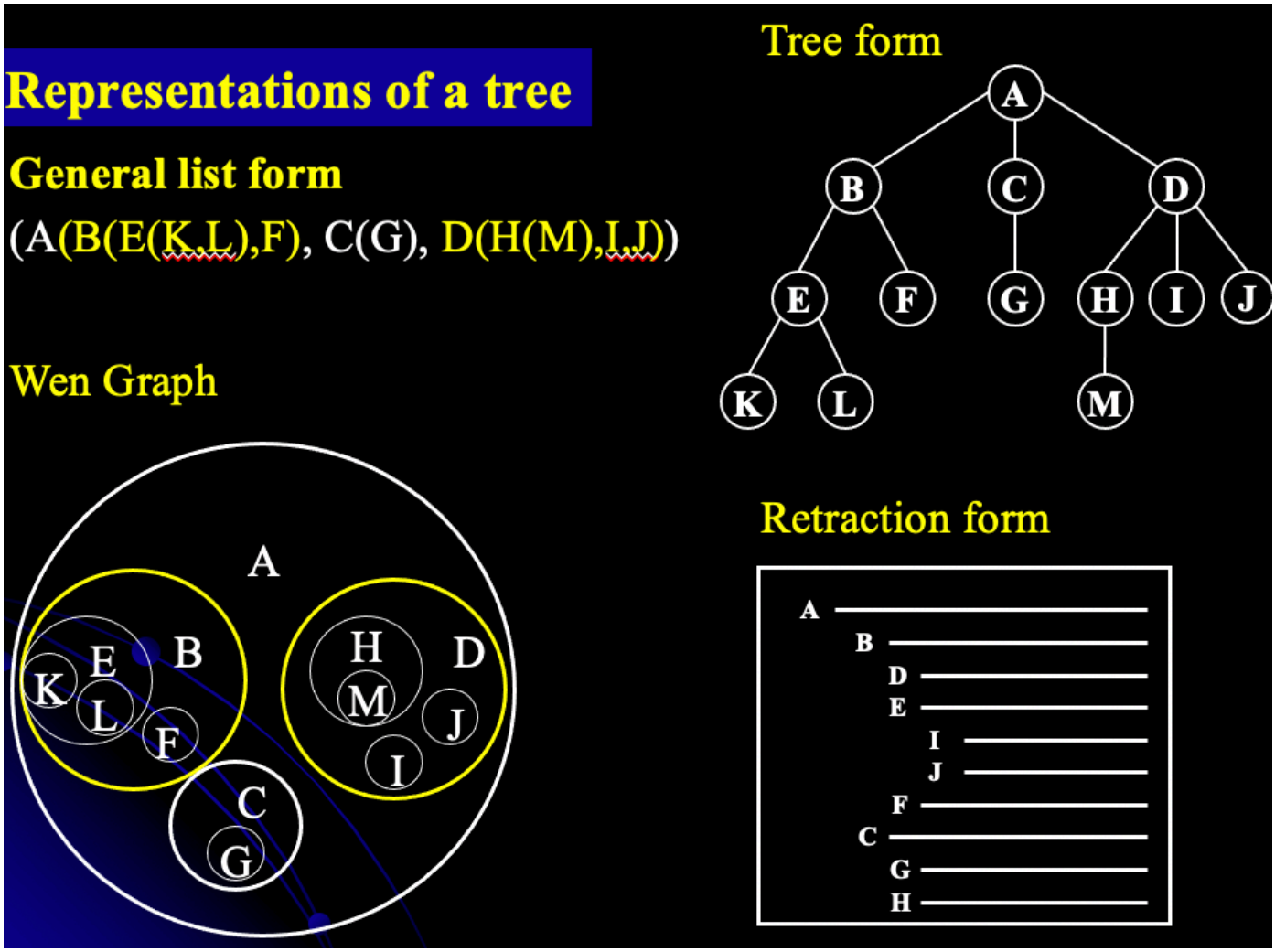
$$RF=\{<root, r_i> \mid i=1,2,\ldots,m, m>0\}$$

这个定义将有助于得到森林和树与二叉树之间转换的递归定义。



# Physic

广义表表示

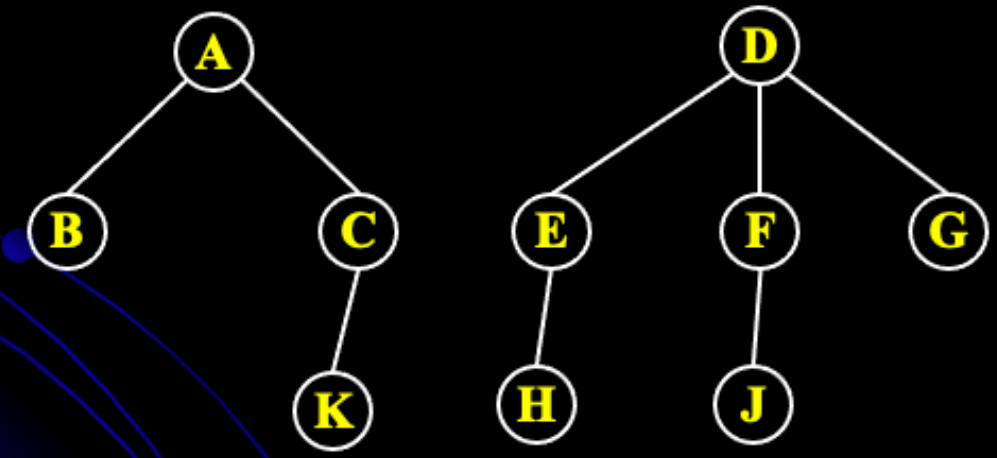



# Algorithm

## 遍历

1. PreOrder 先序　(A, B, C, K, D, E, H, F, J, G)

2. InOrder 中序　(B, K, C, A, H, E, J, F, G, D)




转二叉树

**1. 森林转换成二叉树**

如果$F = \{T_1, T_2, \ldots, T_m\}$是森林，则可按如下规则转换成一棵二叉树$B = \{root, LB, RB\}$。

(1) 若F为空，则B为空树；

(2) 若F非空，则B的根root为森林中第一棵树的根$Root(T_1)$，B的左子树LB是从$T_1$中根结点的子树森林$F_1 = \{T_{11}, T_{12}, \ldots, T_{1m1}\}$转换而成的二叉树；其右子树RB是从森林$F' = \{T_2, T_3, \ldots, T_m\}$转换而成的二叉树。

# 二叉树

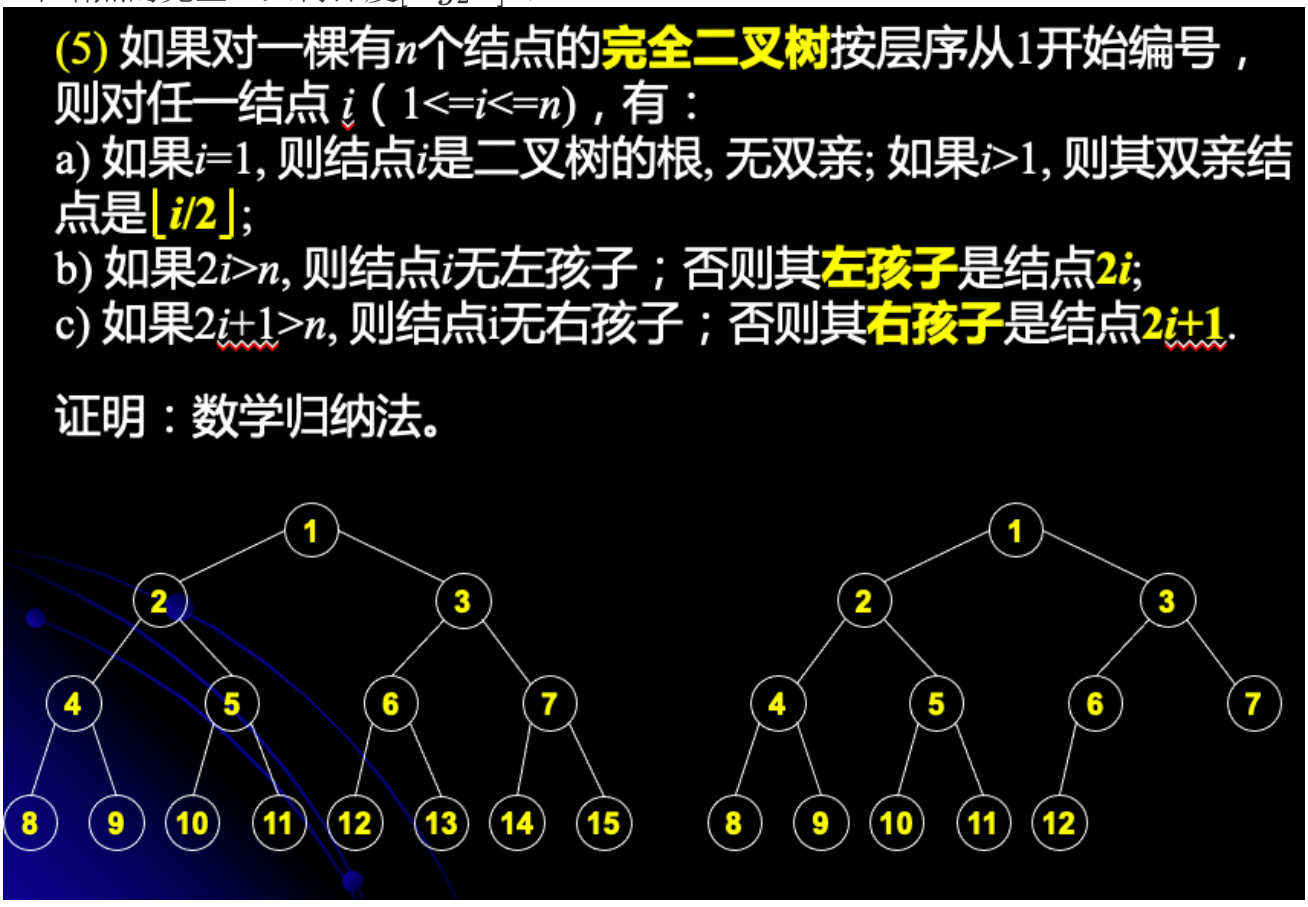
## Logic

每一个结点至多有两棵子树，有左右之分

二叉树和树是两个概念

满二叉树：任何结点或者是树叶，或者有两棵结构完全相同的非空子树

完全二叉树：除最后一层外，每一层的结点均达到最大值，在最后一层上只缺少右边的若干结点

- 在二叉树的第i层上最多有$2^{i-1}$个结点
- 深度为k的二叉树至多有$2^k-1$个结点
- 一棵二叉树，叶子结点树比度为二度结点数多1
- n个结点的完全二叉树深度$\lfloor log_2 n \rfloor + 1$
- 


扩充二叉树：

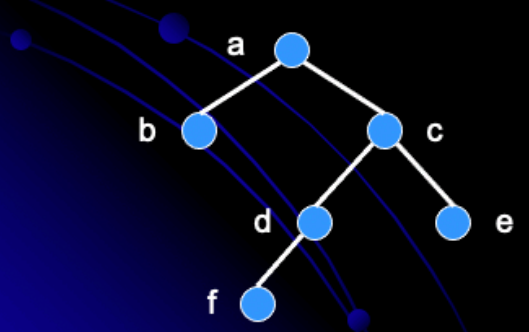
## Extended binary tree (扩充二叉树)：

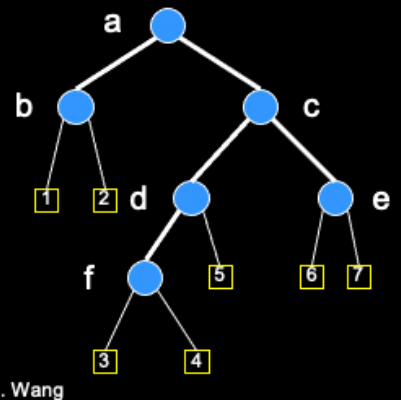
对一个二叉树进行“扩充”，可得到扩充的二叉树。扩充的方法是：

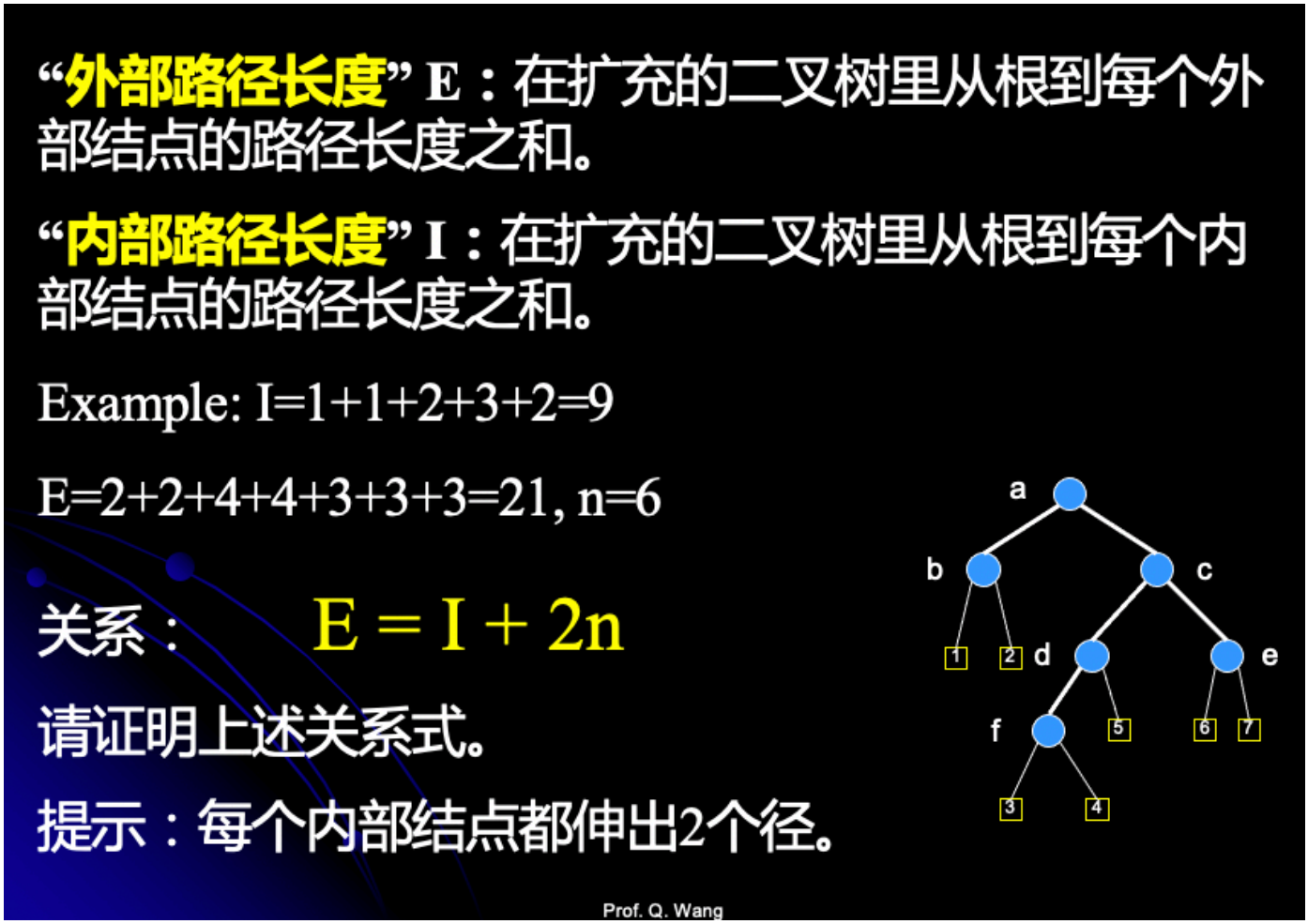
把原二叉树的结点都变为度数为2的分支结点，也就是说，如果原结点的度数为2，则不变，度数为1，则增加一个分支，度数为0(树叶)增加两个分支。

在扩充的二叉树里外部结点 (叶结点)都是新增加的结点。如果原二叉树有$n$个结点，在扩充的二叉树里，这$n$个结点的度数都是2。因此，扩充的二叉树有$2n$条边，$2n+1$个结点，除$n$个原来二叉树结点，还有$n+1$个是新增加的外部结点。

结论：在扩充的二叉树里，新增加的外部结点的个数比原来的内部结点个数多1。



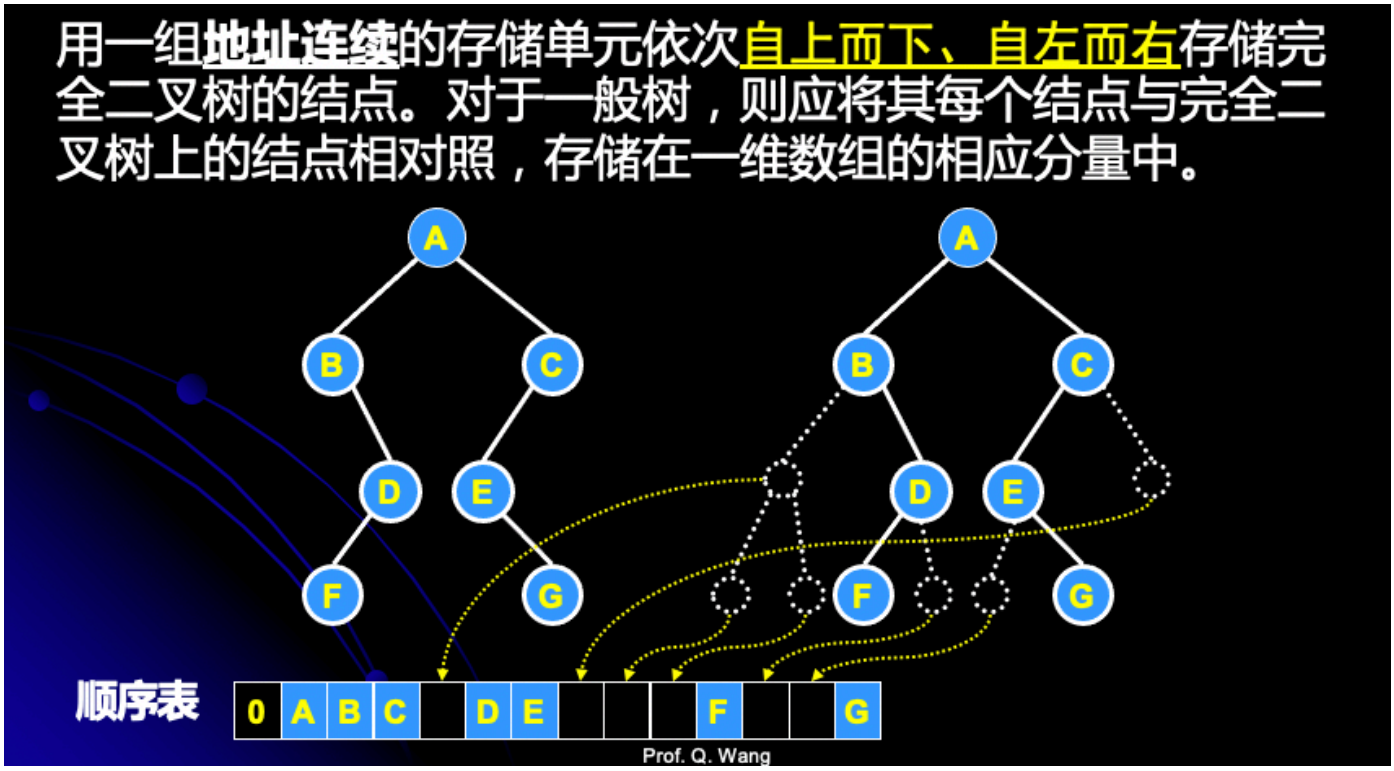






数学归纳法易证

# Physic

## 连续形式

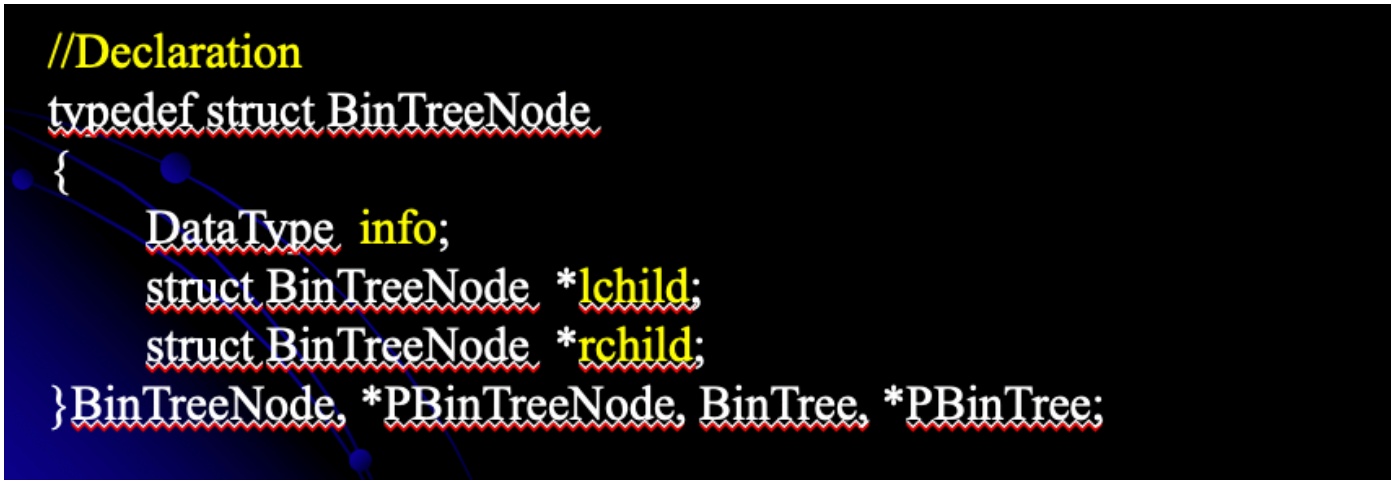



每棵树有一个连续的存储空间和上限空间（改造后个数）

```
//Declaration
#define  MAXNODE  100     /* 定义二叉树中结点的最大个数 */
typedef struct SeqBTree   /* 顺序二叉树类型定义 */
{
      DataType  nodelist[MAXNODE+1];
      int  n;  /* 改造成完全二叉树后，结点的个数 */
}SeqBTree, *PSeqBTree;
```

## 链式形式

二叉链表，数据和左右指针




# Algorithm

## 深度优先遍历

深度优先遍历（从左到右）分为先中后三个顺序，也就是前缀、中缀、后缀（逆波兰表示法）

### 递归算法

按顺序递归访问结点即可 easy pass

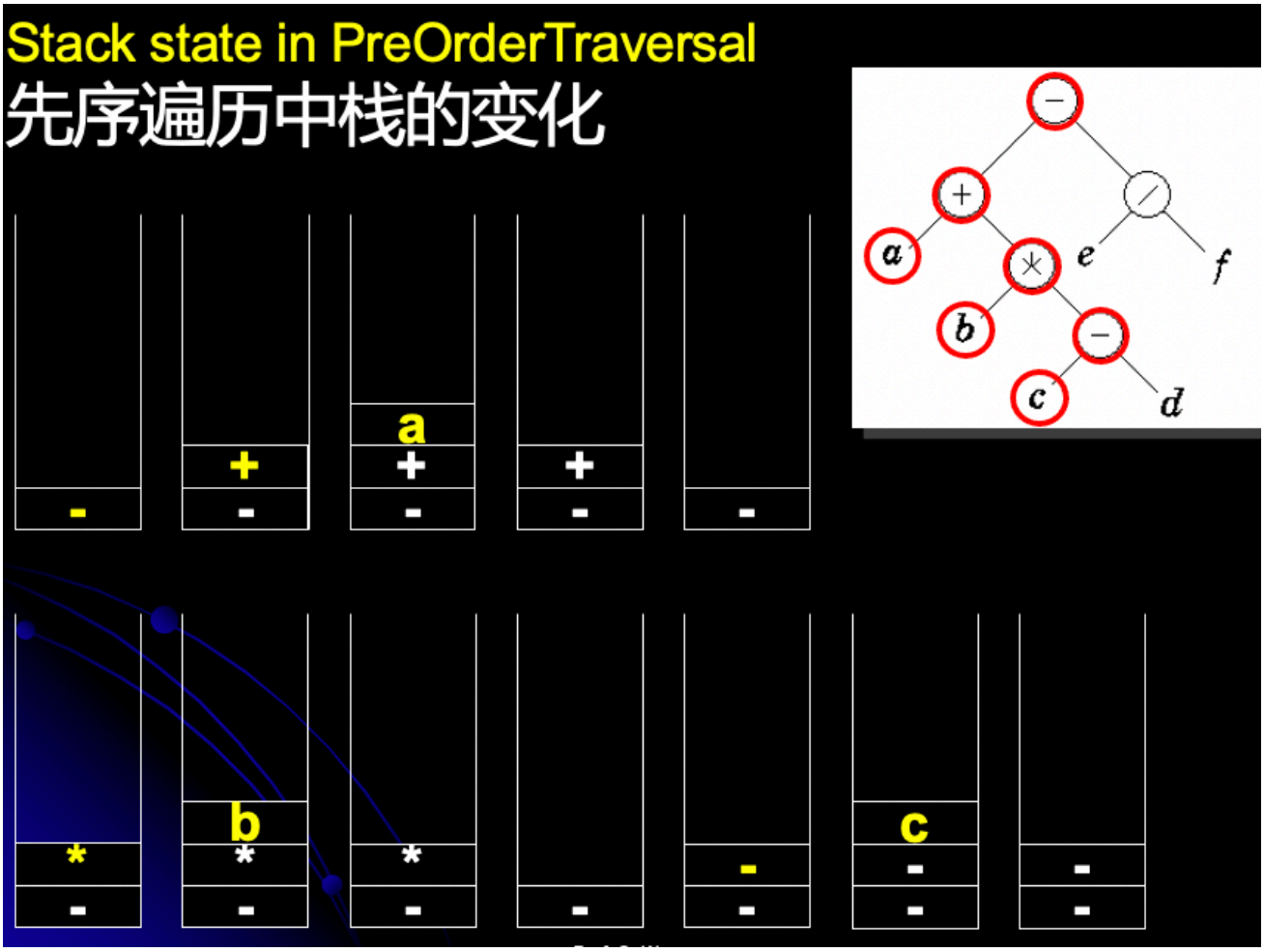
### 非递归算法

先序访问

栈实现

- 先序：遇到一个结点，访问并压栈，然后访问左子树，到底之后栈顶元素退栈，右子树入栈。重复直到栈空树遍历结束

  书上写法：遇到一个结点，访问之，接着把它压入栈中，然后去遍历它的左子树，遍历完它的左子树后，从栈顶弹出这个结点，然后去遍历它的柚子树。



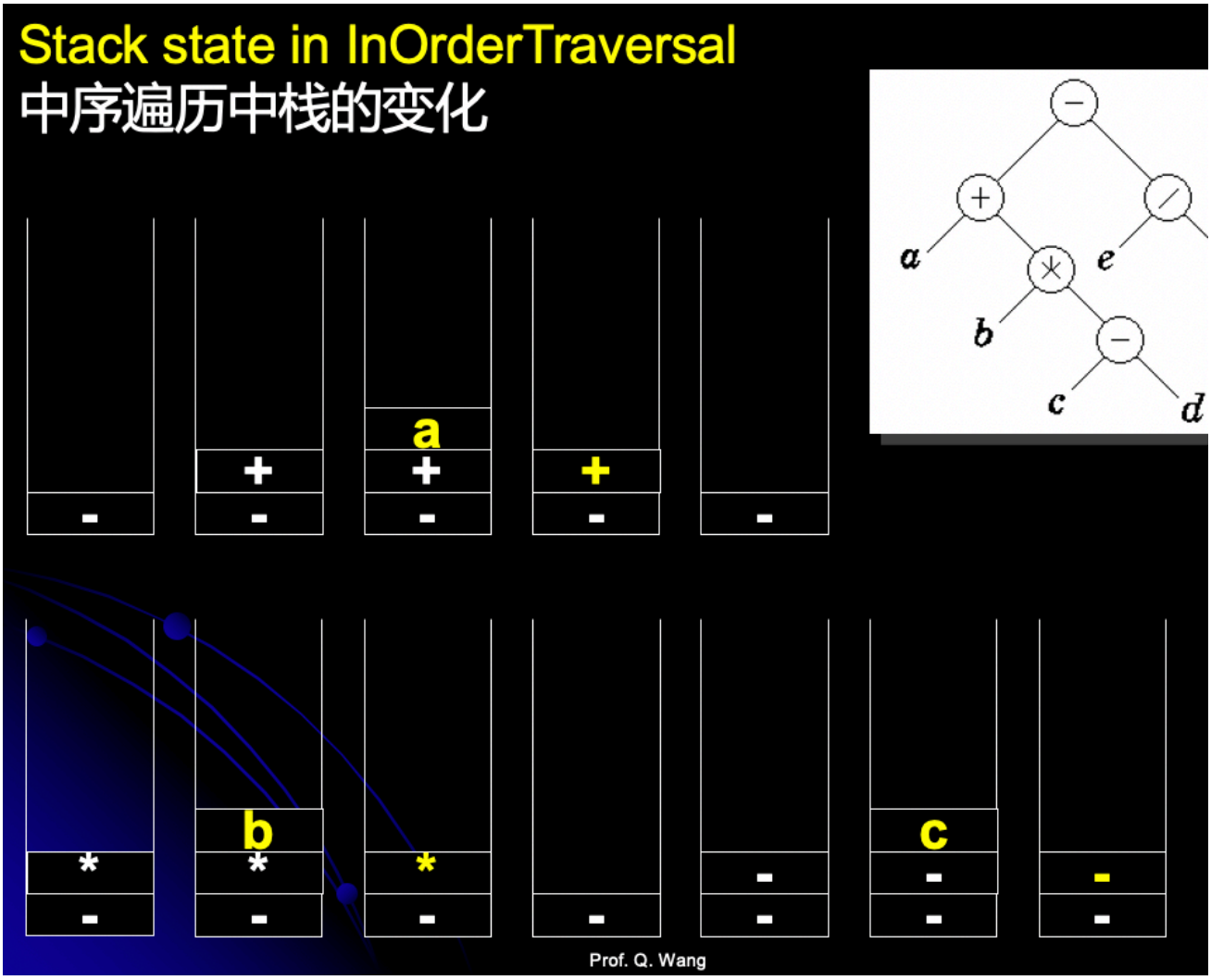

- 优化：什么时候压右子女？仅当该结点的左右子树都不空

中序访问

栈实现

- 和先序类似，也是左子树进栈，但是先不访问左子树，等到左子树到底了，访问栈顶元素，出栈，压入右子女。

书上写法：遇到一个结点，遍历它的左子树，遍历完后，从栈顶弹出这个结点并访问之，然后再去遍历它的柚子树。




后序遍历

栈实现

- 比较复杂，每个结点要等到左右子树都被遍历完成后才被访问，所以在遍历左右子树之前都要进栈，进栈有两个标记，一是刚访问好左子树，马上去访问它的右子树，二是右子树访问完了，可以访问这个结点。
  出栈时，根据标记，进行判断，如果是刚遍历好左子树，把标记更改，二次进栈；如果是刚访问完右子树，则访问当前结点，不再入栈。

- 代码大循环里有两个while，第一个左子树入栈

```
//Postorder Traversal

void PostOrderTraverse (PBinTree T) {
Stack S; PBinTree p;  StackNode w;
   StackEmpty( S );   p = T;
   do {
      while ( p ) {
         w.ptr = p;  w.tag = L;  Push( S, w);
         p = p->lchild;
      }
```



第二个把所有R标记出栈，直到遇到L标记

```
      int continue = 1;
      while ( continue && ! IsEmpty( S ) ) {
         w = getTop( S );  Pop( S );
         p = w.ptr;
         switch (w.tag) {
            case L :  w.tag = R;  Push( S, w);
                      continue = 0;
                      p = p->rchild;
                      break;
            case R :  printf (p->info);
                      break;
         }
      }
   } while ( p || !IsEmpty( S ) );
}
```

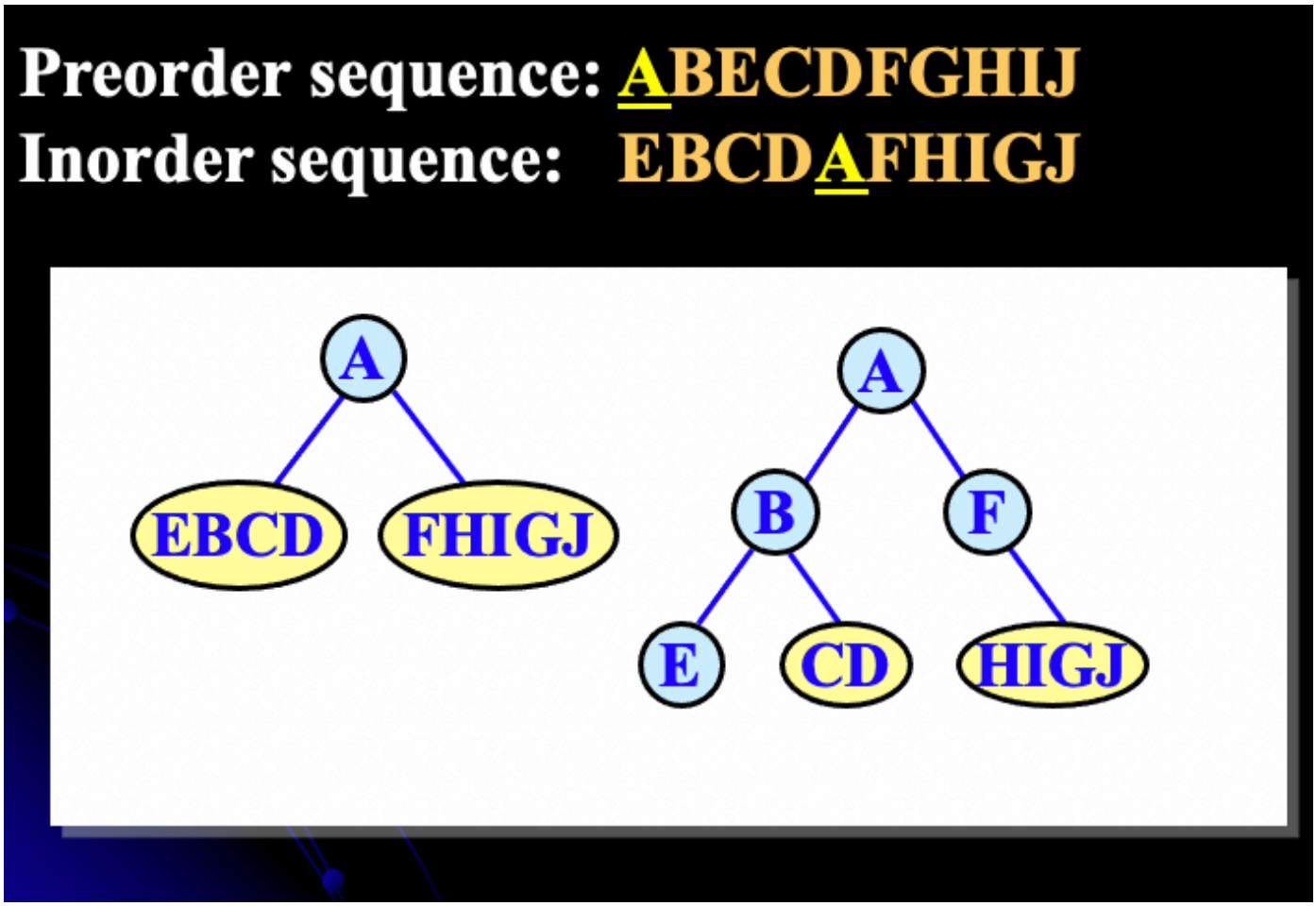
二次进栈

## 广度优先遍历（按层次遍历）

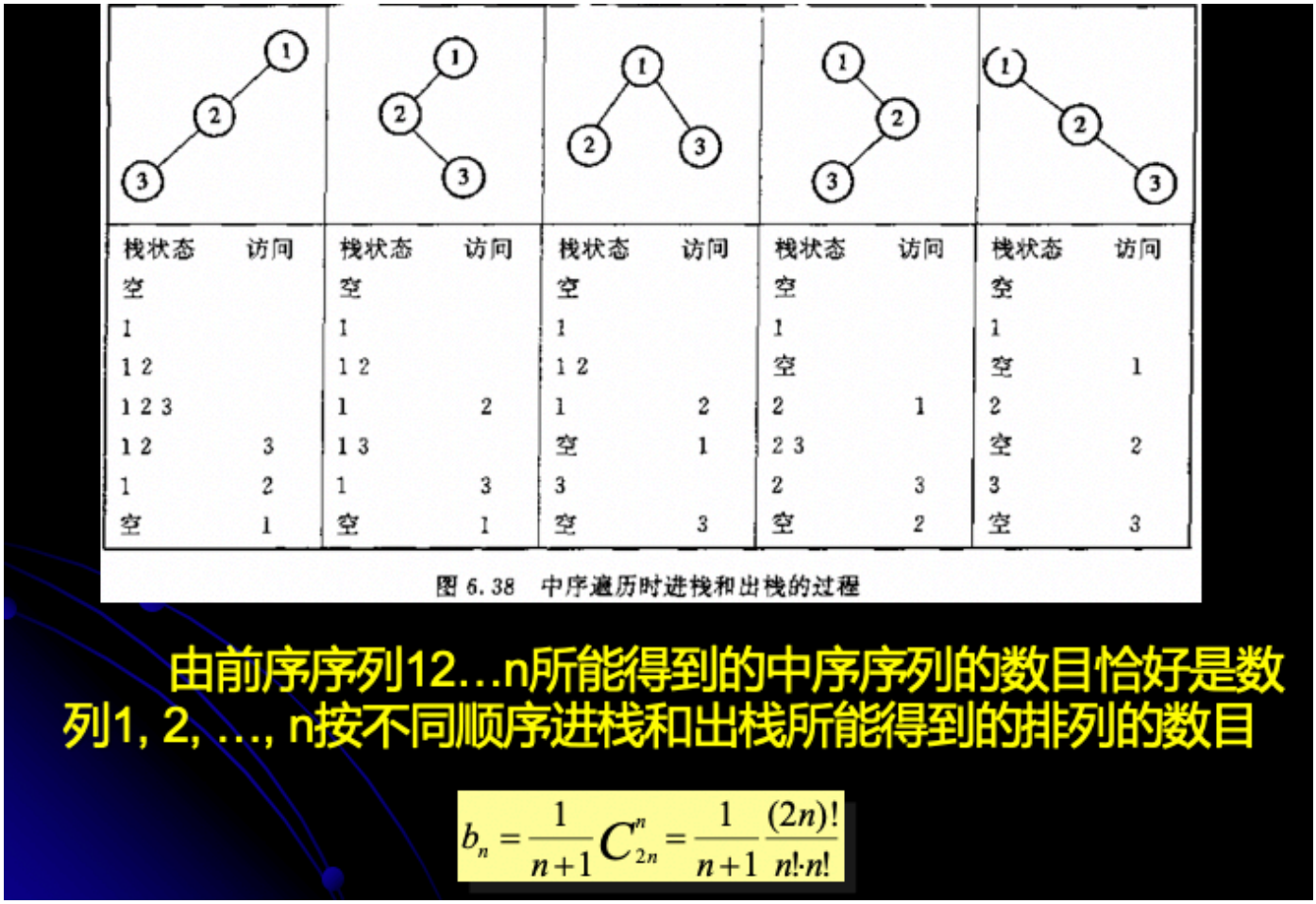
队列实现，先把根入队，然后每次出队时访问结点并把左右孩子入队。

## 二叉树重建

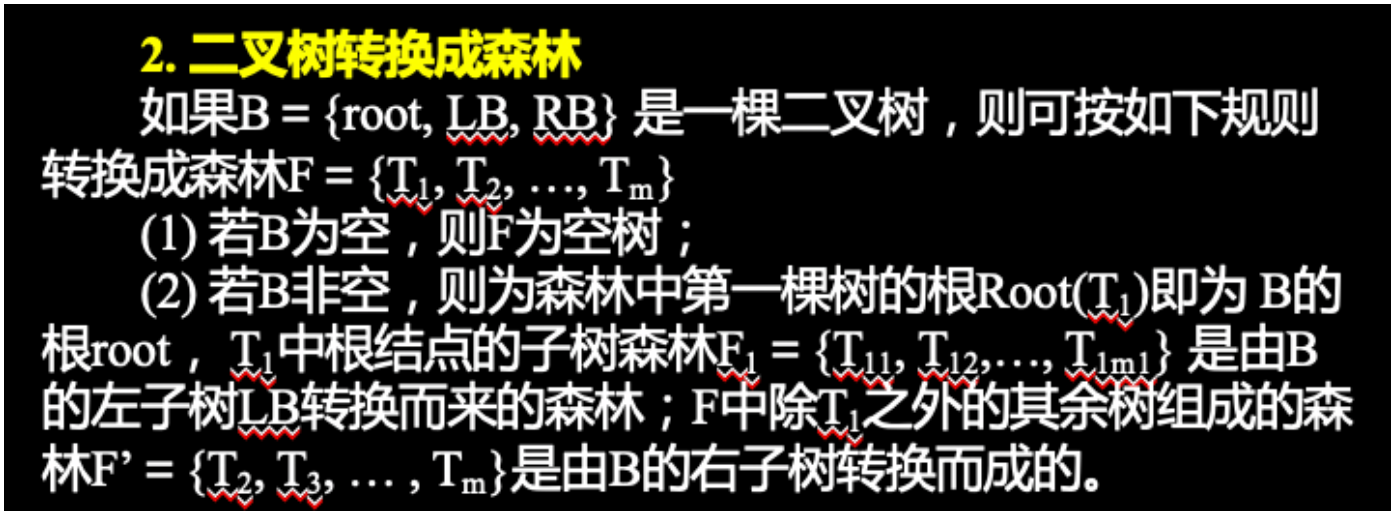
核心思想都一样，对于一个序列和其中的根结点，可以分成这个根结点和它的左右孩子的点集，通过另外一个序列确定这个划分。




## Counting of BT




## 二叉树转森林




## 哈夫曼树

给定一组权，构造带权外部路径最小的扩充二叉树

算法如下：

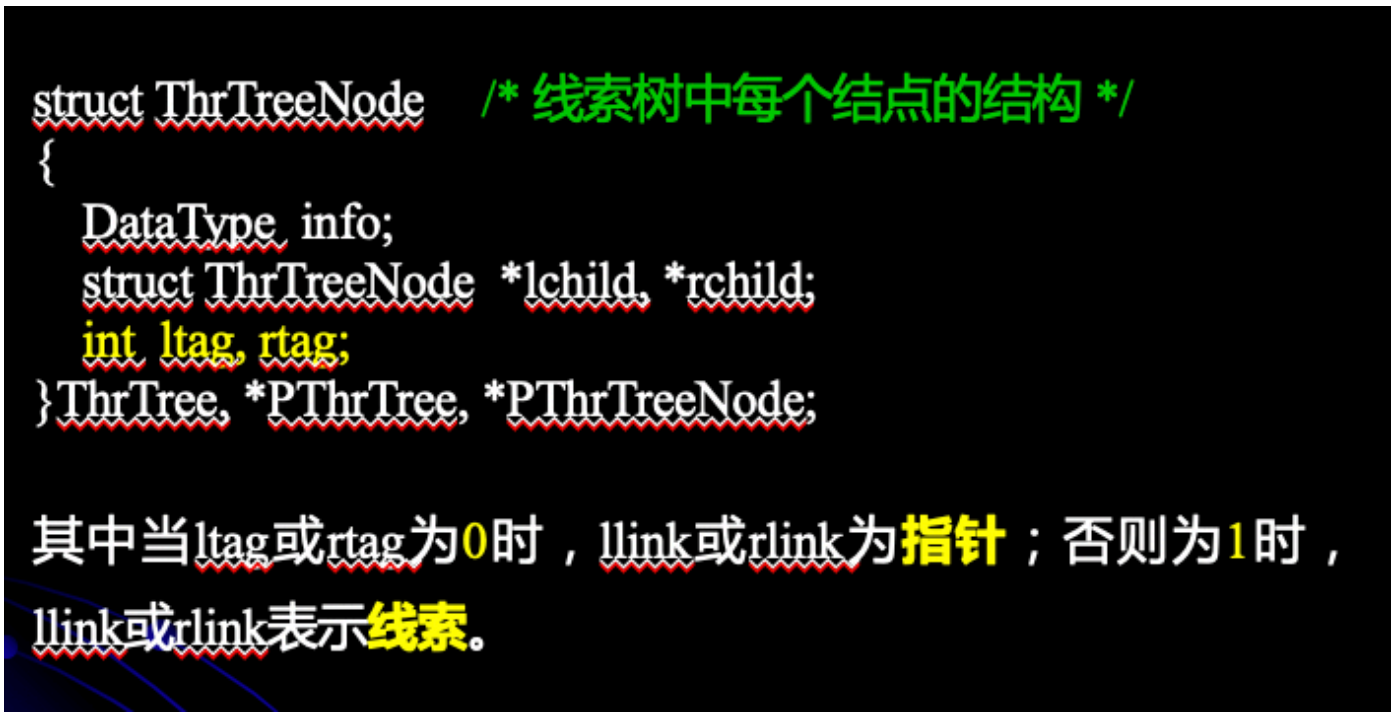
1. 初始时每个权都是一个单独的二叉树
2. 在二叉树集合中选最小的两棵树构成左右子树，新二叉树根的权是和
3. 在二叉树集合里删掉这两个树，把新树放进去

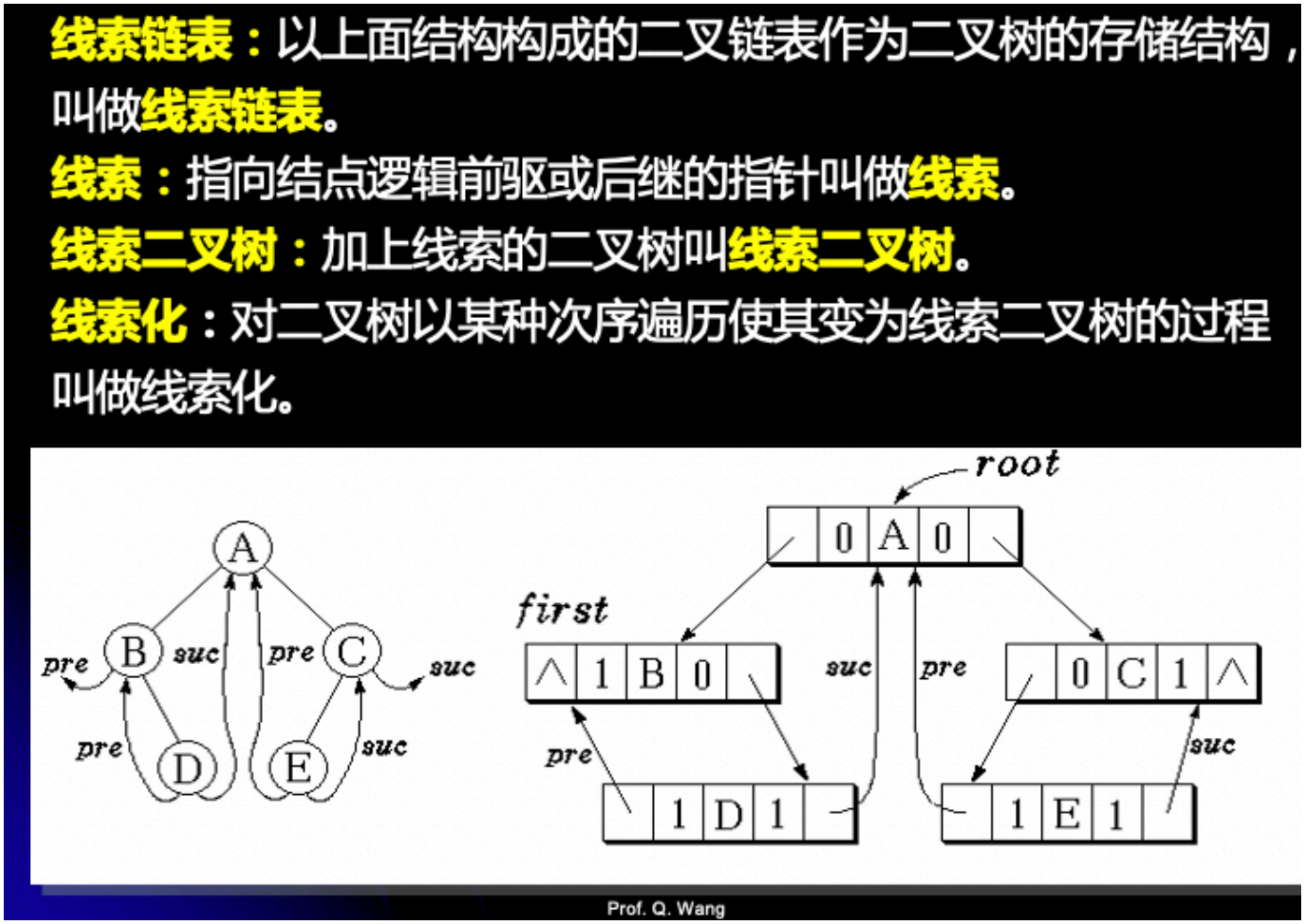
4. 重复至只含一棵树

# 线索二叉树

## Logic&Physci

在任务中，除了物理结构，还想记录任务逻辑中的先后



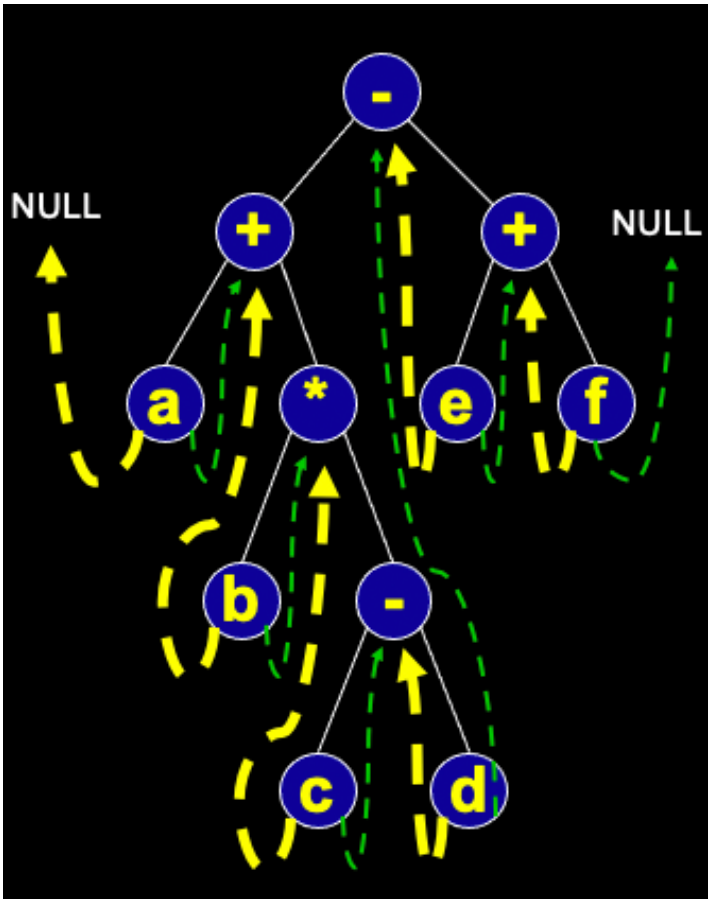



# Algorithm

## 中序线索二叉树

后继：如果右标志为1，则右链为线索，指示后继；否则，后继是柚子树的最左下角的结点（访问右子树时的第一个结点）

前驱：同上，改为左子树的最右下（访问左子树的最后一个）




## 先序线索二叉树

找前驱分为三个情况

1. 结点是树根，则没有前驱
2. 结点是左孩子或唯一的右孩子，前驱为双亲
3. 结点是不唯一的右孩子，则前驱为它双亲左子树上先序遍历的最后一个结点

要知道前驱，就要知道双亲，所以要三叉链表

后继就是左孩子或者唯一的右孩子

## 后序线索二叉树

和上面类似，找后继分三种情况

1. 树根没有后继
2. 右孩子或者唯一的左孩子，后继为双亲
3. 不唯一的左孩子，后继是双亲右子树后序最后一个。

前驱就是右孩子

要用三叉链表

## 中序遍历中序线索二叉树

核心就是找到线索的起点，先顺左子树遍历，直到底或者找到线索，这个时候应该找到了线索的起点。顺着线索遍历，因为后继放在右子树的指针域，所以遍历时查看右子树，如果不是线索就顺着右子树的左子树向下，直到找到线索。

## 在中序线索二叉树里找出指定结点在先根下的后继结点和后根下的前驱结点

原理是，中序线索二叉树的后继是上一个层次的结点

先根的后继

1. 一个结点有左子树，那么它先根的后继是左子树的根
2. 没有左子树，先根的后继是右子树的根
3. ！为叶子时，先根的后继是中序右线索的右子树根

后根的前驱

1. 一个结点有右子树，后根的前驱是右子树的根
2. 没有右子树，后根的前驱是左子树的根
3. ！为叶子时，后根的前驱是中序左线索左子树根

# 图

## Logic

顶点，弧（弧头是左边元素，弧尾是右边元素）。

不考虑顶点到自身的弧和边

n个顶点最多有n(n-1)/2条边，有向图是n(n-1)条弧

网是带权的图

邻接点 度 入度 出度

简单路径：不重复出现的路径

回路/环：第一个顶点和最后一个顶点相同的路径

简单回路/环

有根图：从一个顶点能到任意顶点

连通图：对于五香兔而言

连通分量（极大连通子图）

强连通图：有向图

强连通分量

生成树：连通图的极小连通子图，n-1条边，加一条边就有环

如果一个有向图恰有一个顶点入度为0，其余顶点入度均为1，则该图必定是一棵有向树。所有树可以看成是图的特例。

生成森林：由若干棵有向树组成，含有图中所有顶点，只有构成不相交的有向树的弧。

关节点：删去某个顶点和与他相连的边后，图的一个连通分量变成两个。

重连通图：没有关节点

连通度：至少删去多少个结点才会破坏连通性

最长的路径：关键路径

# Physic

## 邻接矩阵

```
Declaration of adjacent matrix
typedef struct
{
    VexType vexs[MAXVEX];                /* 顶点信息 */
    AdjType arcs[MAXVEX][MAXVEX];        /* 邻接矩阵信息 */
    int   arcCount, vexCount;            /* 图的顶点个数 */
}Graph, *PGraph;
```

## 邻接表

对每一个顶点，建立一个单链表存依附于它的弧或边，顶点还是一个一维数组。

每一个顶点有两个域，一个顶点的info域，一个以这个顶点为弧尾的顶点的相邻点链表。

```
typedef struct
{       VexType vertex;                  /* 顶点信息 */
        PEdgeList edgelist;              /* 边表头指针 *
} VexNode;                               /* 顶点表 */
```

相邻点链表和顶点一起，存放了这个顶点发出的弧，它的结构是

```
typedef struct EdgeNode
{       int   endvex;                    /* 相邻顶点字段 */
        AdjType  weight;                 /* 边的权 */
        struct EdgeNode  *nextedge;      /* 链字段 */
}EdgeList, *PEdgeList, *PEdgeNode, EdgeNode;   /* 边表 */
```

而整个图，就是这样的一系列结点的集合

```
typedef struct
{       VexNode vexs[MAXVEX];
        int vexNum, edgeNum;                    /* 图的顶点和边个数
}GraphList;
```

## 十字链表

每个顶点都知道自己的入弧链表和出弧链表

```
typedef struct _VexNode
{
        VexType  vertex;
        ArcNode  *firstIn;      /* 指向该顶点的第一条入弧*/
        ArcNode  *firstOut;     /* 指向该顶点的第一条出弧*/
}VexNode;
```

弧链表结点知道起止点，还知道同尾/同头的下一条弧

```
Declaration of arc node
typedef struct _ArcNode
{
        int       tailVex, headVex;     /* 弧的头尾顶点的位置*/
        struct _ArcNode *hLink;         /* 弧头相同的弧的链域*/
        struct _ArcNode *tLink;         /* 弧尾相同的弧的链域*/
        AdjType  weight;
}ArcNode;
```

图里是一个顶点集合

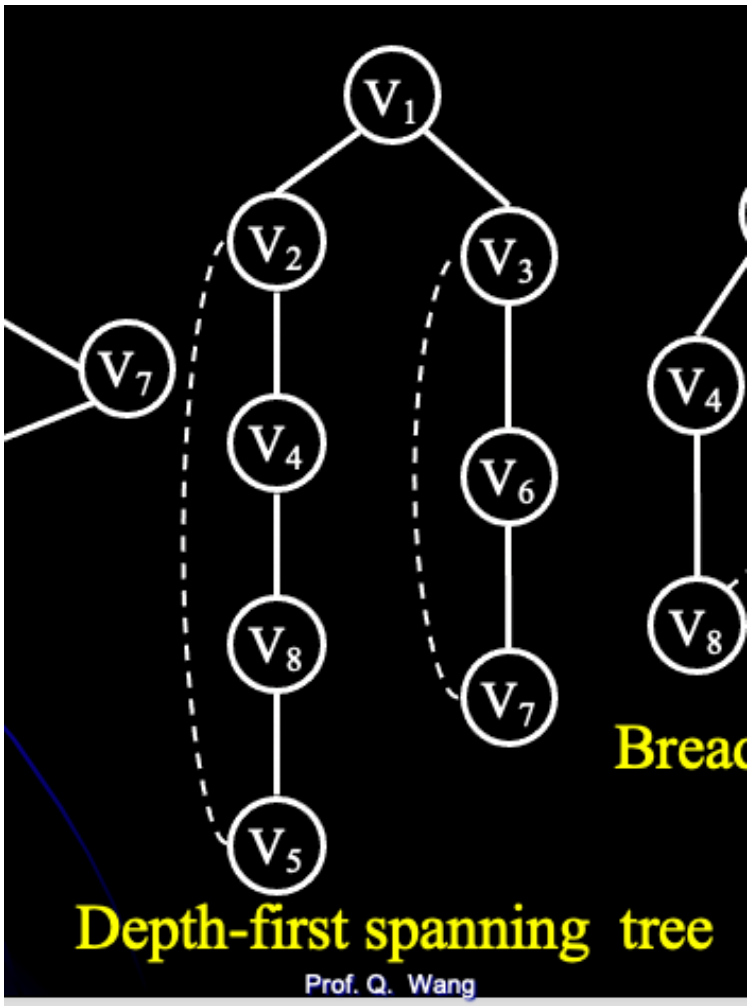
**Declaration of cross linked list of directed graph**

```
typedef struct
{
        VexNode  xList[Max_Vert_Num];     /* 表头向量*/
        int  vexNum, arcNum;
}OLGraph;
```

# Algorithm

## DFS

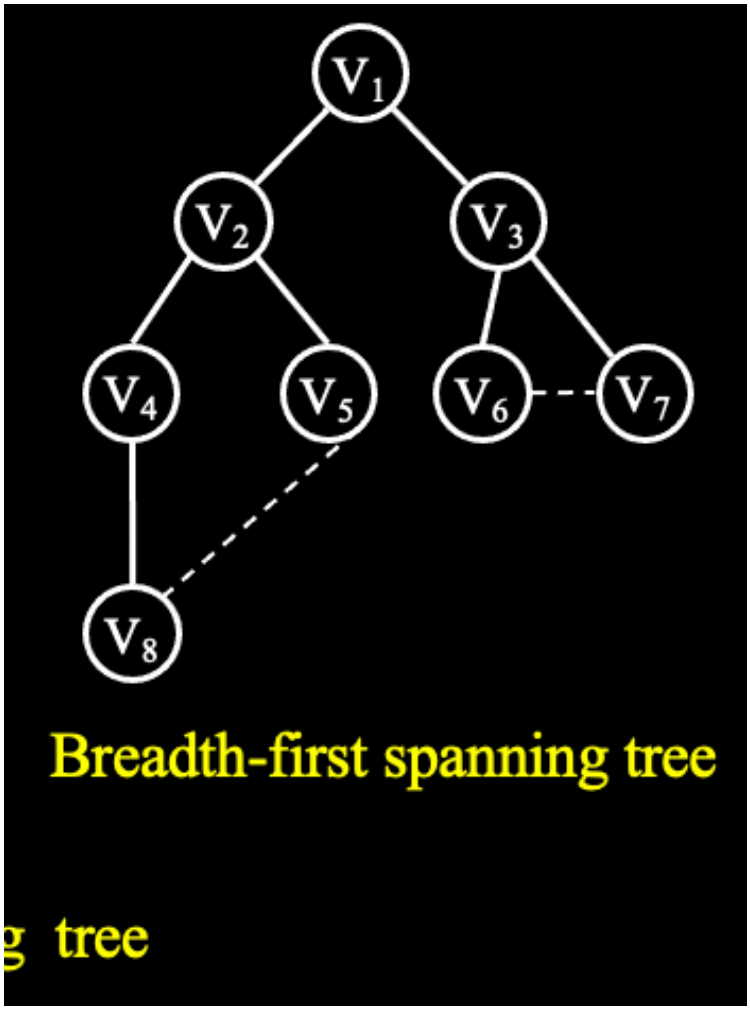
找到底后回溯，easy pass，深度优先生成树



## BFS

用队列，easy pass

广度优先生成树

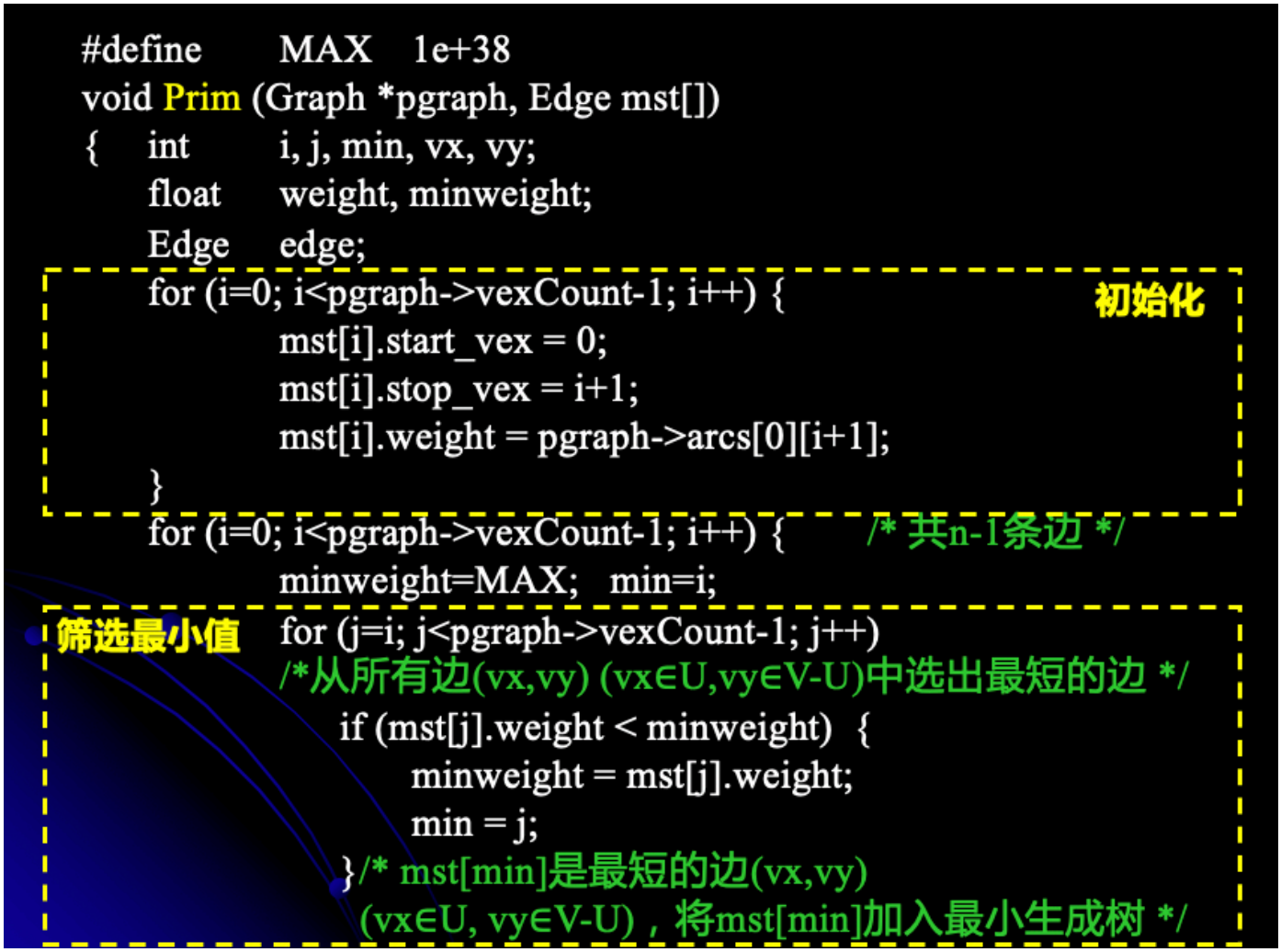



## 最小生成树的Prim算法

初始找一个点放到结果点集合U里，V-U就是剩下n-1个点的集合，找一条从U到V-U的代价最小的路，把V-U里的那个结点拿到U里，重复n-1次。

写代码有讲究，初始化mst之后筛选更新，每次更新都能保证新的边一定在mst里选，因为mst里的每条边mst[i]代表从U到V-U中的i点的最短路。

```
#define      MAX   1e+38
void Prim (Graph *pgraph, Edge mst[])
{    int      i, j, min, vx, vy;
     float    weight, minweight;
     Edge     edge;
     for (i=0; i<pgraph->vexCount-1; i++) {                    初始化
               mst[i].start_vex = 0;
               mst[i].stop_vex = i+1;
               mst[i].weight = pgraph->arcs[0][i+1];
     }
     for (i=0; i<pgraph->vexCount-1; i++) {      /* 共n-1条边 */
               minweight=MAX;   min=i;
筛选最小值     for (j=i; j<pgraph->vexCount-1; j++)
               /*从所有边(vx,vy) (vx∈U,vy∈V-U)中选出最短的边 */
                    if (mst[j].weight < minweight)  {
                         minweight = mst[j].weight;
                         min = j;
                    }/* mst[min]是最短的边(vx,vy)
                     (vx∈U, vy∈V-U)，将mst[min]加入最小生成树 */
```




## 最小生成树的Kruskal算法

初始时点集U里只有点没有边，每次添加一条最短边，要求边的两个端点在不同的连通分量上，否则舍弃

## 最短路的Dijkstra算法

所有点到某一点到最短路

1. 初始化一个数组D[n-1],其中D[i]代表arcs[0] [i]
2. 找最小的D[j]
3. 用D[j]+arcs[j] [i]试探更新D[i]
4. 重复23n-1次

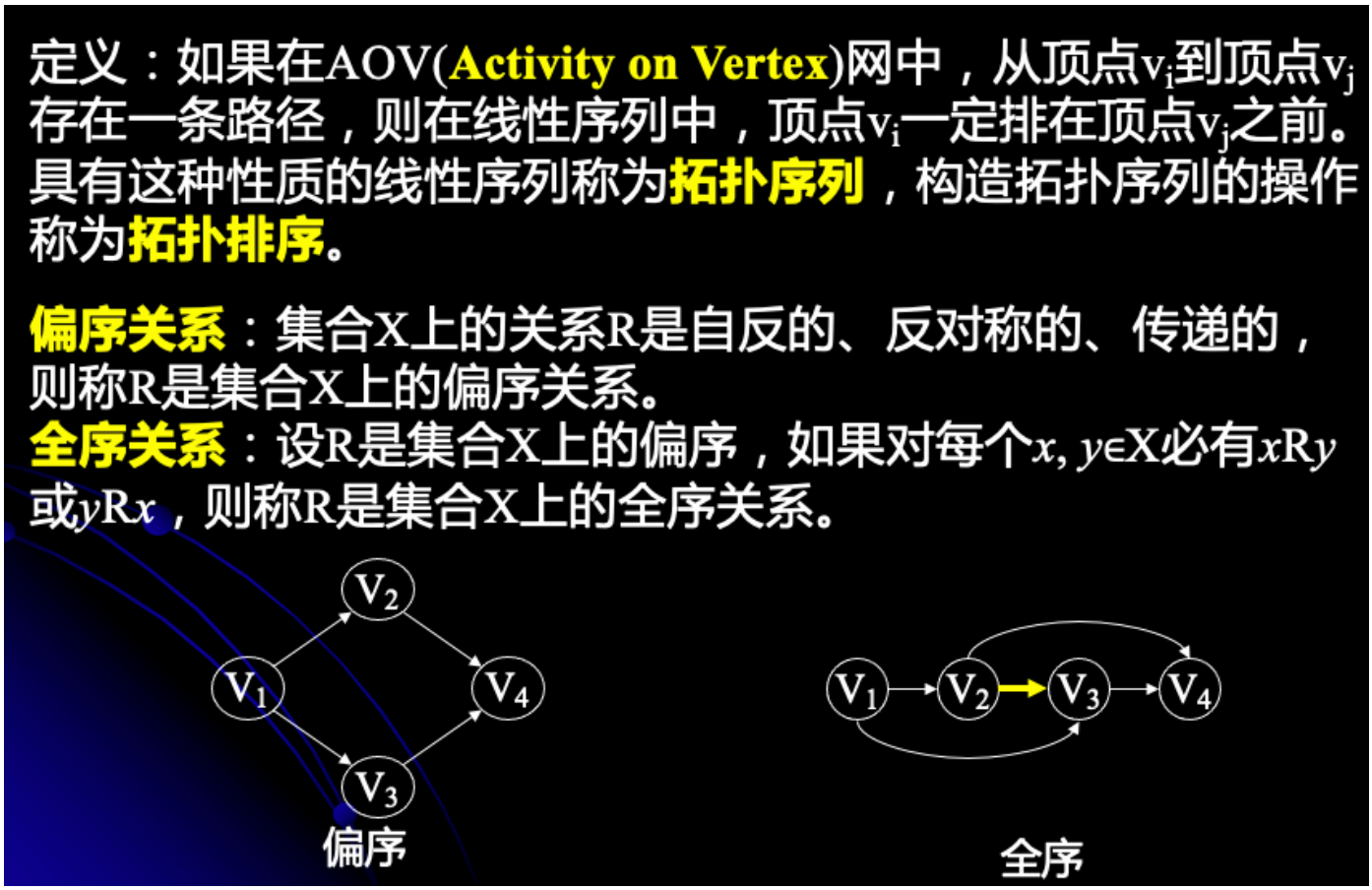
## 最短路的Floyd算法

任意两点的最短路

1. 初始化a[m] [n]为arcs[m] [n]
2. (Vi,v0,vj)如果存在，试探更新a[i] [j]（找到中间结点序号不大于零的最短路）
3. (Vi,...v1,...vj)如果存在，试探更新a[i] [j]（找到中间结点序啊后不大于1的最短路）

4. 重复比较n次，得到结果

## 拓扑排序




算法：

1. 从图中选一个没前驱的点输出删除
2. 删掉以这个点为尾的弧
3. 重复12

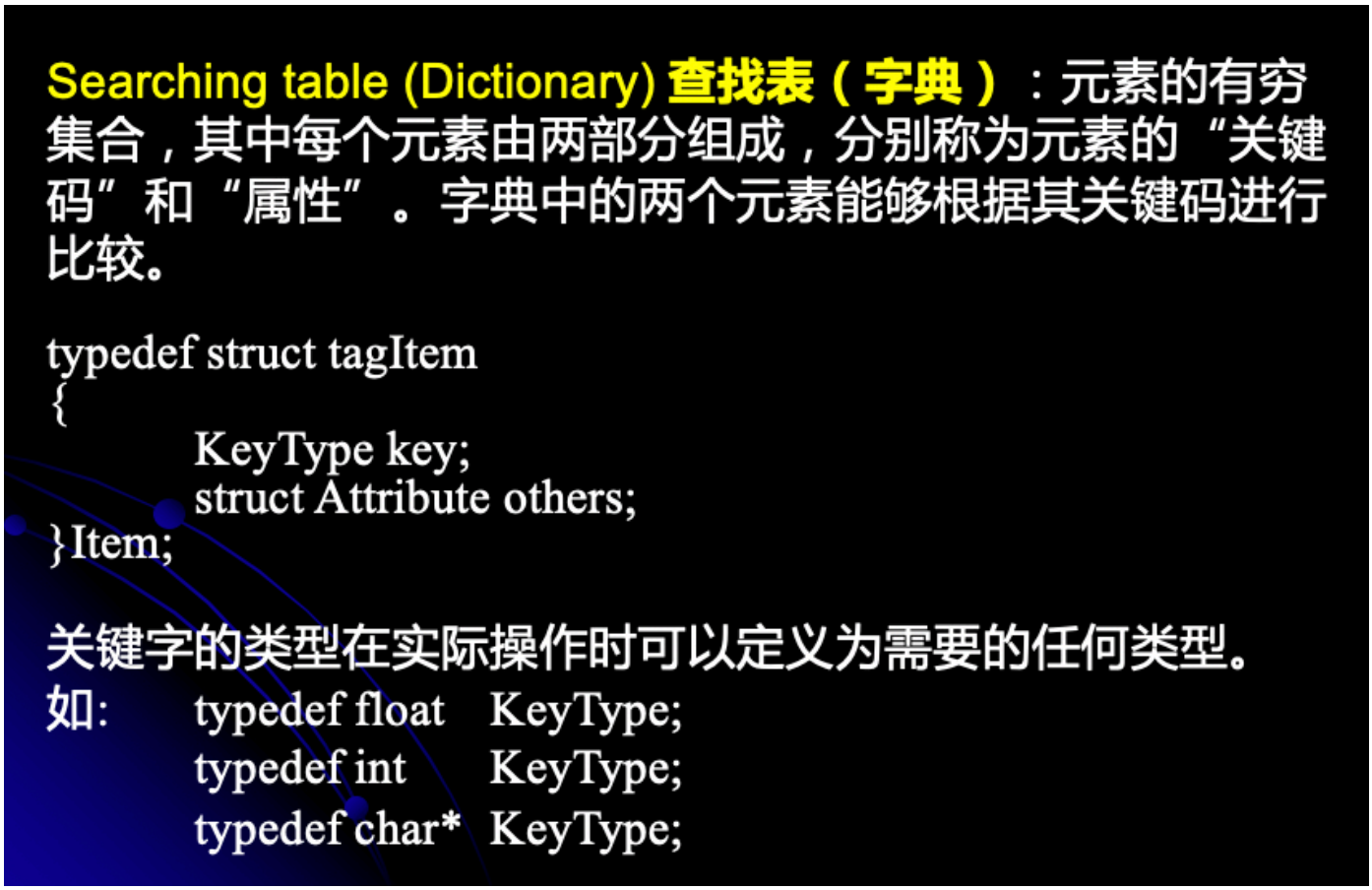
## 逆拓扑排序

找出度为0的结点

1. 从图中选一个没后继的点输出删除
2. 删掉以这个点为头的弧
3. 重复12

# 查找表

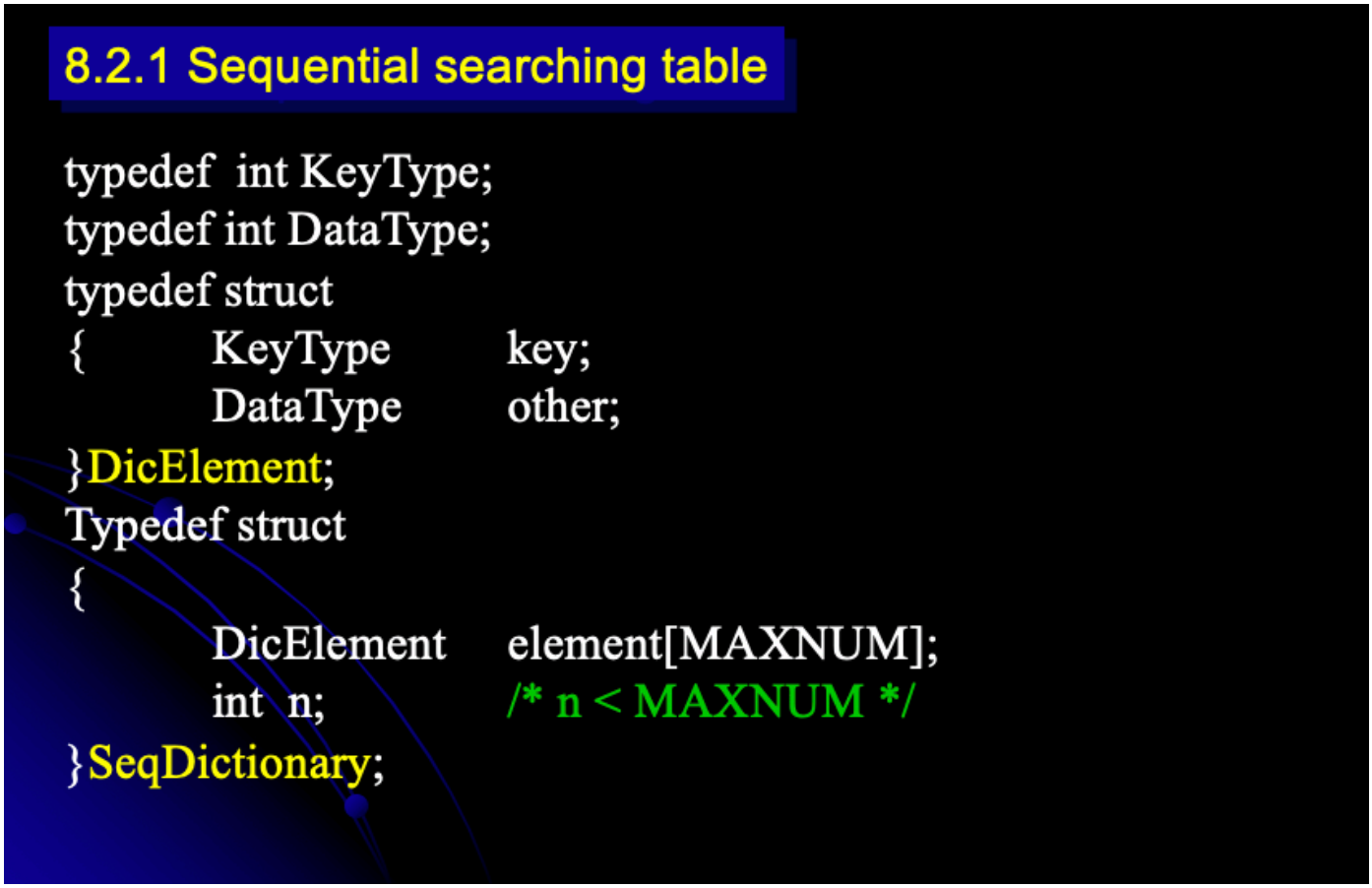
## Logic




## Physic&Algorithm

### 静态查找表

顺序




平均查找长度

$$\frac{1}{2}\frac{1}{n}\sum_{i=1}^{n}(n-i+1)+\frac{1}{2}(n+1)=\frac{3}{4}(n+1) \tag{1}$$

## 二分查找

有序的顺序表

$$\frac{n+1}{n}log_2(n+1)-1 \tag{2}$$

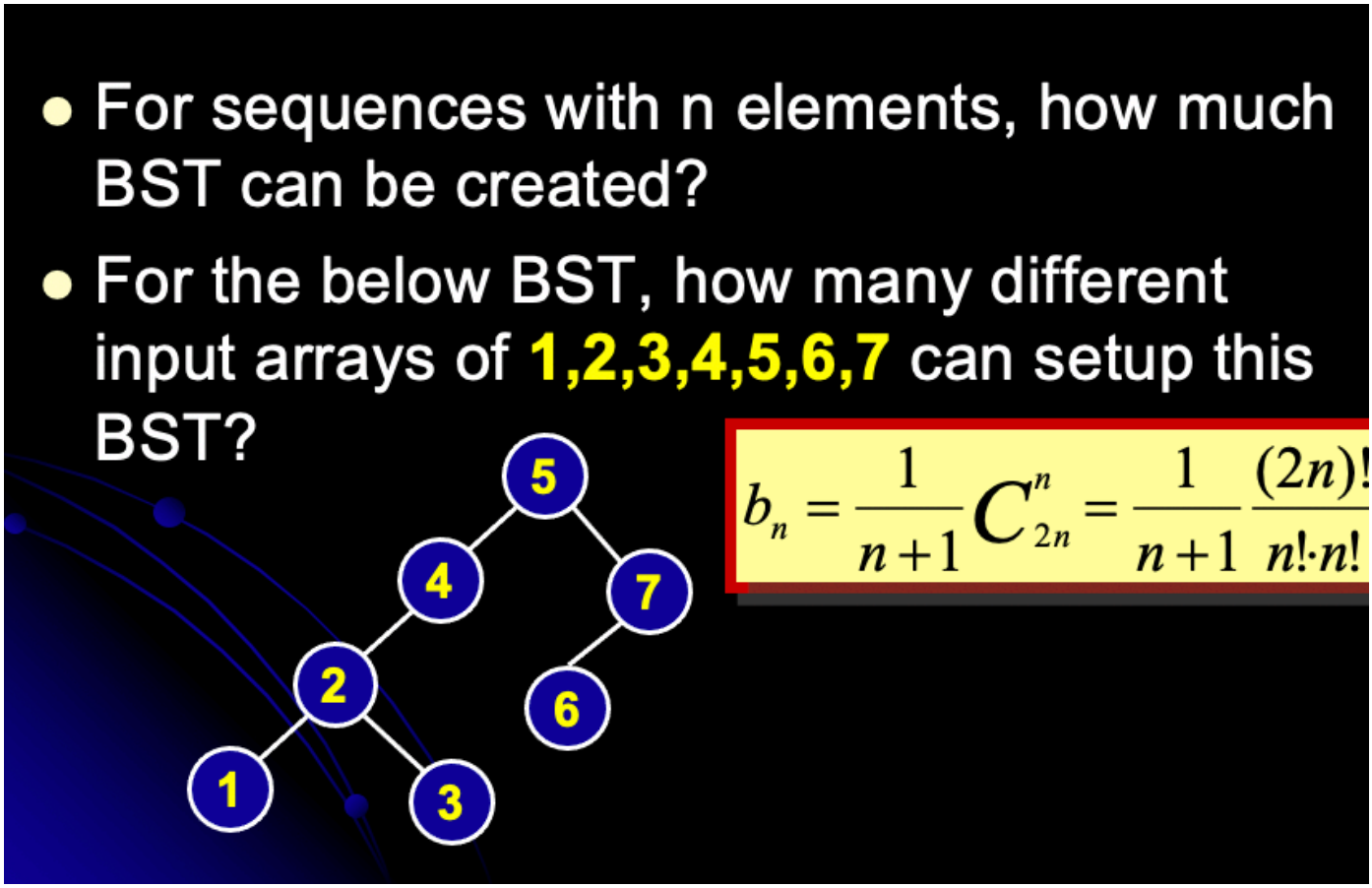
## 二叉排序树

不空的话

- 左子树不空则左子树上所有值都比根小
- 右子树不空则右子树上所有值都比根大
- 左右子树也是二叉排序树

插入算法（意味着在某个结点上会失败了）：

1. 若排序树空，把这个数值放在根
2. 若不空，向当前结点左右子树插入，小于向左，大雨向右
3. 左右子树插入原则相同




平均查找长度

$$2(\frac{n+1}{n})log_e n \tag{3}$$

## 平衡二叉树

要么是空树，要么左右子树都是平衡二叉树，且左右子树深度之差的绝对值不超过一

平衡因子：左子树深度减右子树深度

深度和$log_2 N$同量级

如何平衡?

假设由于在二叉排序树上插入结点而失去平衡的最小子树根结点的指针为a（即a是离插入结点最近，且平衡因子绝对值大于1的祖先结点），则失去平衡后进行调整的规律可归纳为下列四种情况：

A. **单向右旋**平衡处理：在*a的左子树根结点的左子树上插入结点。如Fig.a

B. **单向左旋**平衡处理：在*a的右子树根结点的右子树上插入结点。如Fig.b

C. **双向旋转（先左后右）**平衡处理：在*a的左子树根结点的右子树上插入结点。如Fig.c

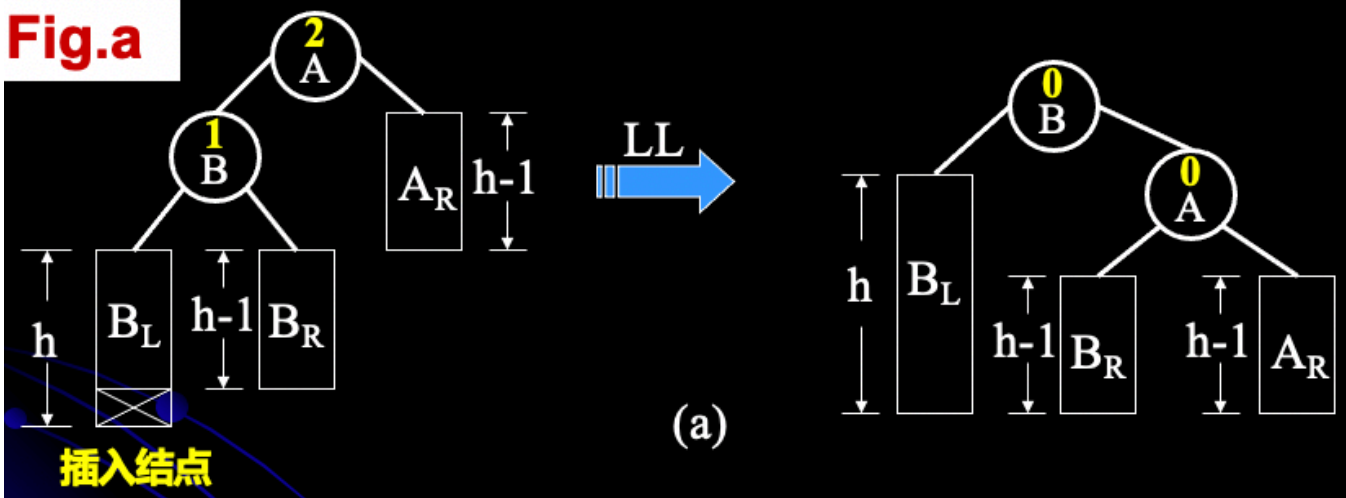
D. **双向旋转（先右后左）**平衡处理：在*a的右子树根结点的左子树上插入结点。如Fig.d

上面的几种情况在经过平衡旋转处理后，以*b或*c为根的新子树为平衡二叉树，而且**它的深度和插入之前以*a为根的子树相同**。因此，当平衡的二叉排序树因插入结点而失去平衡时，仅需对最小不平衡子树进行旋转处理即可。

Go

# 单向右旋

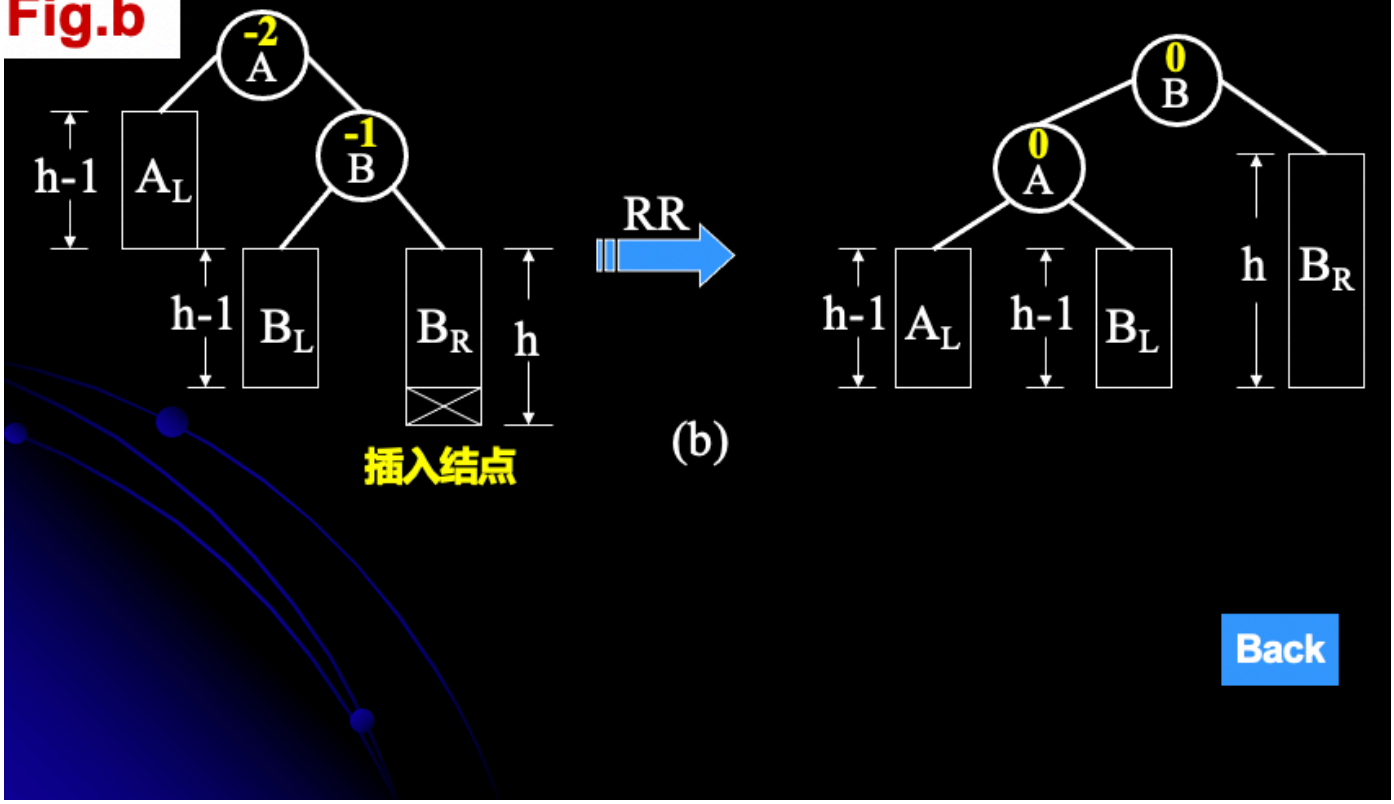
Fig.a



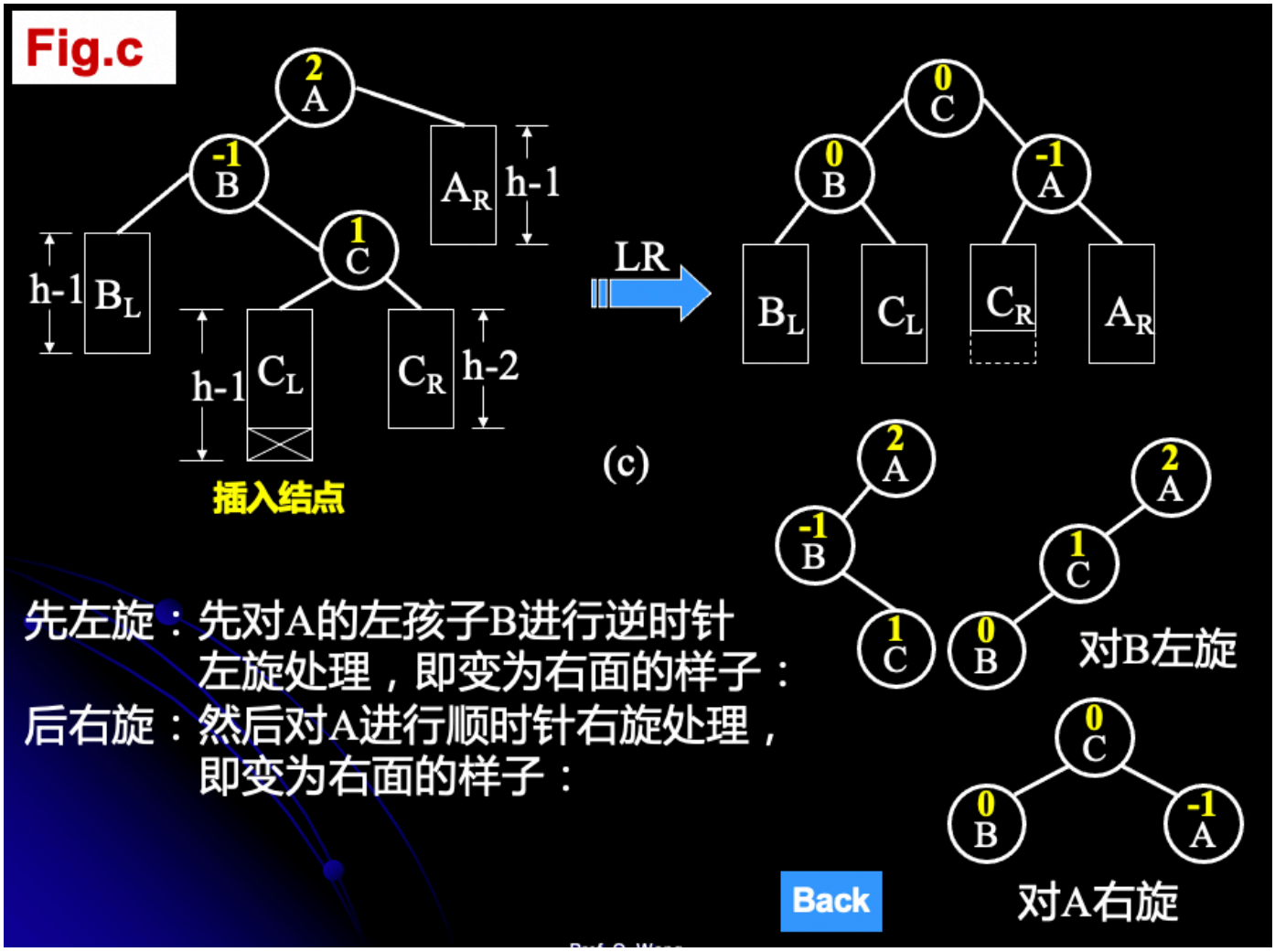

Back

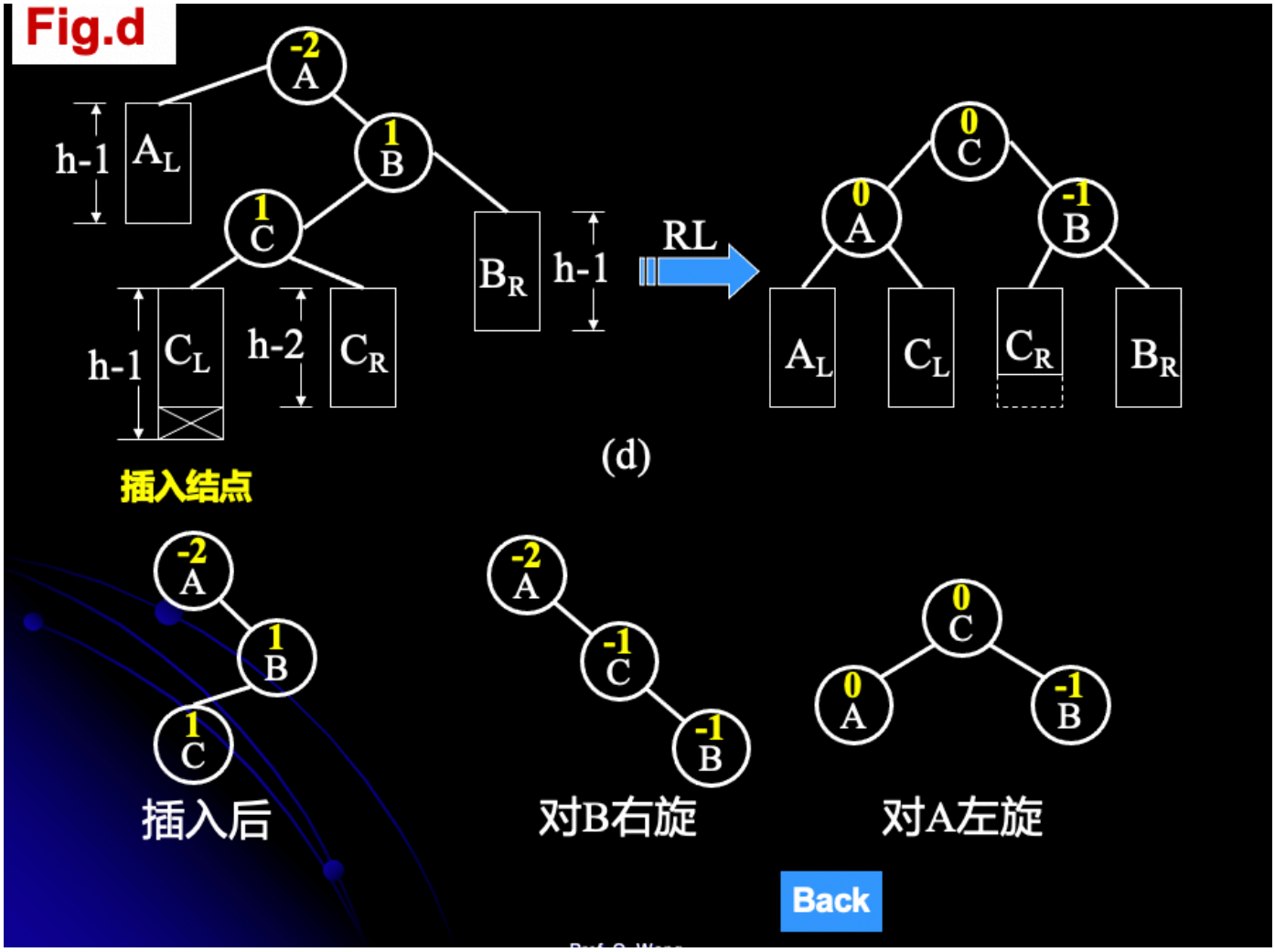
# 单向左旋

Fig.b




Back


Fig.c
2
A
-1
B
1
C
$A_R$
h-1
h-1
$B_L$
h-1
$C_L$
$C_R$
h-2
插入结点
LR
0
C
0
B
-1
A
$B_L$
$C_L$
$C_R$
$A_R$
(c)
2
A
-1
B
1
C
2
A
1
C
0
B
对B左旋
0
C
0
B
-1
A
对A右旋
先左旋：先对A的左孩子B进行逆时针
左旋处理，即变为右面的样子：
后右旋：然后对A进行顺时针右旋处理，
即变为右面的样子：
Back




平衡二叉树查找时间复杂度

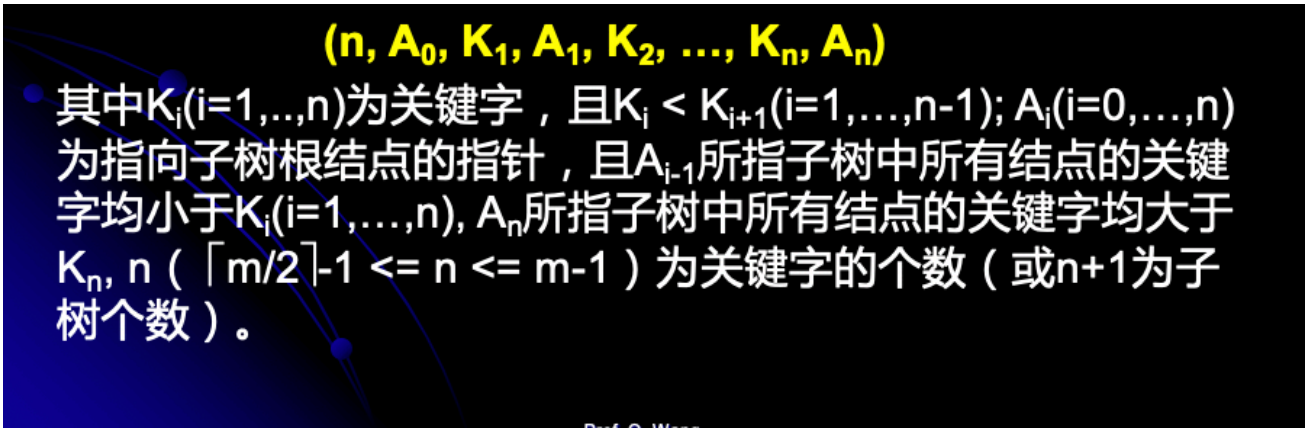
$$O(logn) \tag{4}$$

## B树

高维数据检索和文件索引

m阶B树或为空树，或

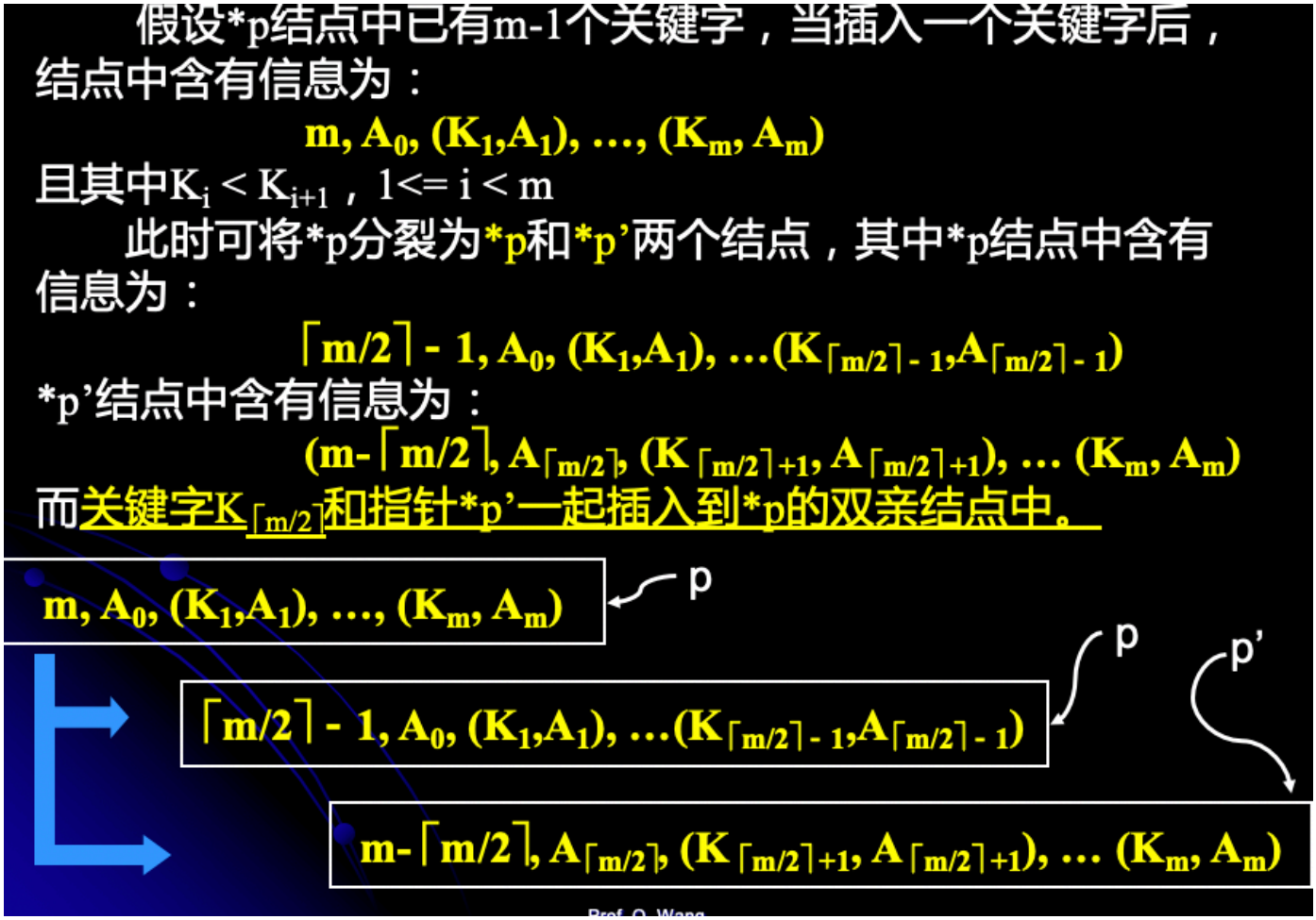
- 每个结点至多有m棵子树
- 根结点若不是叶子，则至少有两棵子树
- 除根之外的所有非终端结点至少有[m/2]棵子树
- 非终端结点包括




- 所有叶子都在一层，且不带信息（实际上为空，指向他们的不存在）

m=2时，B树就是二叉排序树

当插入后某个结点的关键字达到m时，可以分裂成两个结点



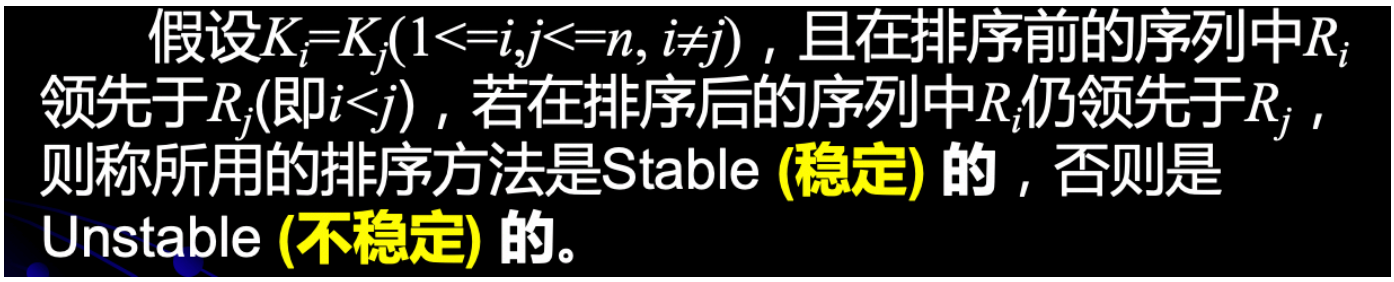

## 散列表

哈希表，要求一个好的哈希函数和一个处理冲突的方法

# 排序

## Logic

稳定性




排序必须要比较关键字，可能要移动数据

## Algorithm

### 插入排序

只要一个空间辅助

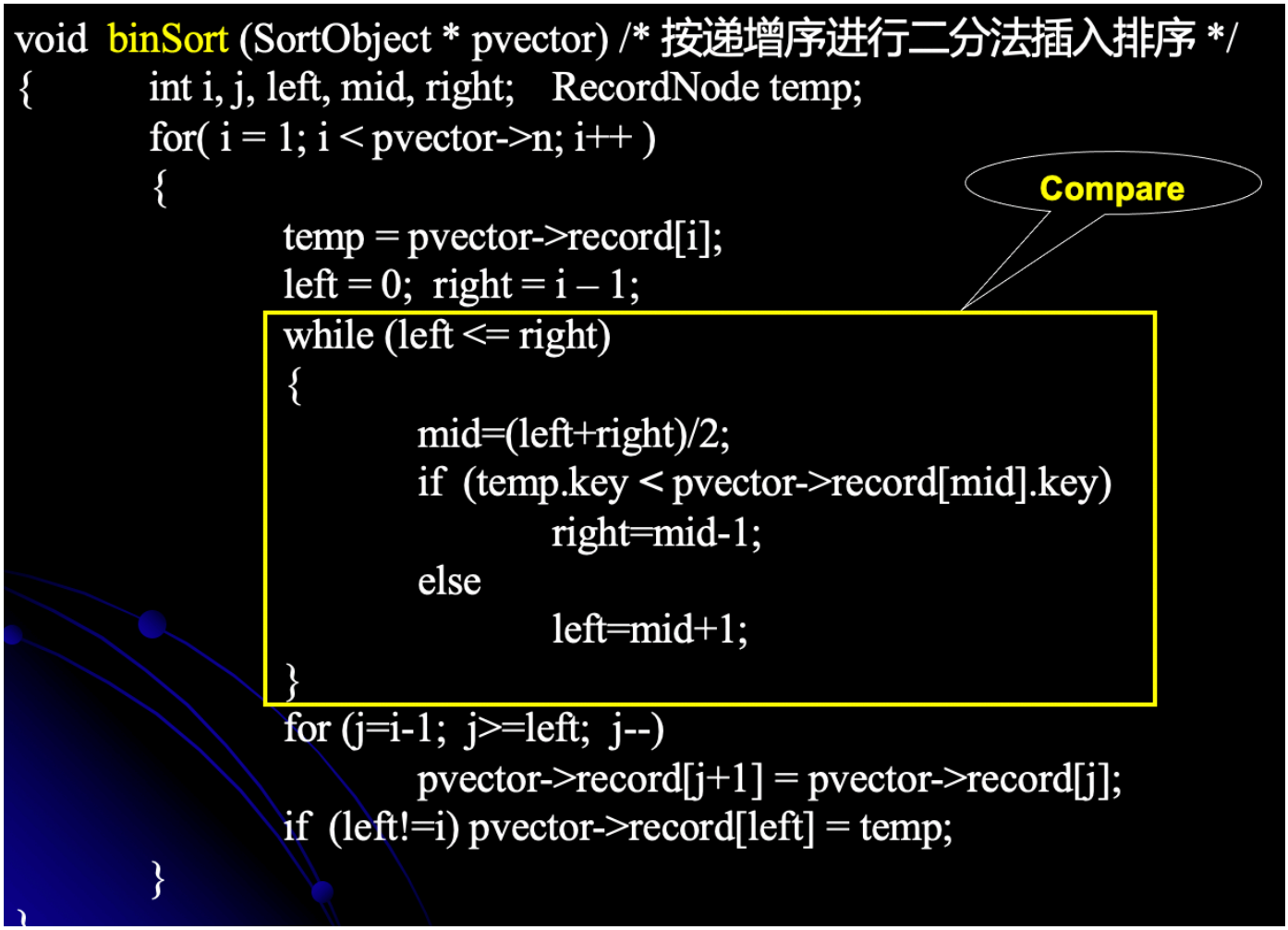
时间:




稳定

### 二分插入排序

每次插入的时候从排好序的序列中间开始插，稳定




## 二路插入排序

用一个辅助数，小于它的一边，大于它的一边

n个辅助存储空间

- 在2-路插入排序中，移动记录的次数约为$n^2/8$。
- 2-路插入排序需要$n$个辅助存储空间。

- 注意：pvector[1]若是待排序记录中关键字最小或最大的记录，2-路插入排序就**退化**为直接插入排序。

冒泡排序

- 在对象的初始排列已经按关键码从小到大排好序时，此算法只执行一趟起泡，做 $n$-1 次关键码比较，不移动对象。这是最好的情形。
- 最坏的情形是算法执行了$n$-1趟起泡，第 $i$ 趟做了 $n$-$i$ 次关键码比较，执行了$n$-$i$ 次对象交换。这样在最坏情形下总的关键码比较次数KCN和对象移动次数RMN为：

$$KCN = \sum_{i=1}^{n-1}(n-i) = \frac{1}{2}n(n-1)$$

$$RMN = 3\sum_{i=1}^{n-1}(n-i) = \frac{3}{2}n(n-1)$$

一个附加空间

稳定

## 快速排序

- 可以证明，函数*quicksort*的平均计算时间也是**O($n\log_2 n$)**。实验结果表明：就平均计算时间而言，快速排序是我们所讨论的所有内部排序方法中最好的一个。
- 快速排序是递归的，需要有一个栈存放每层递归调用时的指针和参数。
- 最大递归调用层次数与递归树的深度一致，理想情况为$\lceil \log_2(n+1) \rceil$。因此，快速排序的存储开销为 **O($\log_2 n$)**。

不稳定，n小慢

## 选择排序

- 直接选择排序的时间复杂度：

  移动：最好时：0　　最坏时：$3(n-1)$

  比较：$n(n-1)/2$

  总的时间复杂度：$O(n^2)$
- 稳定性：**不稳定**

## 堆排序

其比较次数为：$< 2n(\lfloor \log_2 n \rfloor)$. 在最坏的情况下，时间复杂度也是$O(n\log n)$。且仅需一个记录大小的辅助空间。

归并排序

- 时间复杂度：$O(n\log_2 n)$
- 空间复杂度：和待排记录等量的空间
- 特点：是一种**稳定**的排序方法。但是，一般情况下很少用于内部排序。

| 排序方法 | 最坏时间复杂度 | 平均时间复杂度 | 辅助空间 | 稳定性 |
|---|---|---|---|---|
| 直接插入排序 | $O(n^2)$ | $O(n^2)$ | $O(1)$ | 稳定 |
| 二分插入排序 | $O(n^2)$ | $O(n^2)$ | $O(1)$ | 稳定 |
| Shell排序 | $O(n^{1.3})$ | $O(n^{1.3})$ | $O(1)$ | 不稳定 |
| 直接选择排序 | $O(n^2)$ | $O(n^2)$ | $O(1)$ | 不稳定 |
| 堆排序 | $O(n\log_2 n)$ | $O(n\log_2 n)$ | $O(1)$ | 不稳定 |
| 冒泡排序 | $O(n^2)$ | $O(n^2)$ | $O(1)$ | 稳定 |
| 快速排序 | $O(n^2)$ | $O(n\log_2 n)$ | $O(\log_2 n)$ | 不稳定 |
| 归并排序 | $O(n\log_2 n)$ | $O(n\log_2 n)$ | $O(n)$ | 稳定 |
| 基数排序 | $O(d*(n+rd))$ | $O(d*(n+rd))$ | $O(rd+n)$ | 稳定 |